3-D-ANAS: 3-D Asymmetric Neural Architecture Search for Fast Hyperspectral Image Classification

Haokui Zhang, Chengrong Gong, Yunpeng Bai, Zongwen Bai, and Ying Li

***Abstract*— Hyperspectral images (HSIs) provide abundant spectral and spatial information, playing an irreplaceable role in land-cover classification. Recently, based on deep learning (DL) technologies, an increasing number of HSI classification approaches have been proposed, which demonstrate promising performance. However, previous studies suffer from two major drawbacks: 1) the architecture of most DL models is manually designed, relies on specialized knowledge, and is relatively tedious. Moreover, in HSI classifications, datasets captured by different sensors have different physical properties. Correspondingly, different models need to be designed for different datasets, which further increases the workload of designing architectures and 2) the mainstream framework is a patch-to-pixel framework. The overlap regions of patches of adjacent pixels are calculated repeatedly, which increases computational cost and time cost. In addition, the classification accuracy is sensitive to the patch size, which is artificially set based on extensive investigation experiments. To overcome the issues mentioned above, we first propose a 3-D asymmetric neural network search algorithm and leverage it to automatically search for efficient architectures for HSI classifications. By analyzing the characteristics of HSIs, we specifically build a 3-D asymmetric decomposition search space, where spectral and spatial information is processed with different decomposition convolutions. Furthermore, we propose a new fast classification framework, i.e., pixel-to-pixel classification framework, which has no repetitive operations and reduces the overall cost. Experiments on three public HSI datasets captured by different sensors demonstrate the networks designed by our 3-D asymmetric neural architecture search (3-D-ANAS) achieve competitive performance compared to several state-of-the-art methods, while having a much faster inference speed. Code is available at: https://github.com/hkzhang91/3D-ANAS.**

***Index Terms*— Asymmetric network search, asymmetric search space, hyperspectral image (HSI) classification, inference speed, patch-to-pixel framework, pixel-to-pixel framework.**

Manuscript received January 11, 2021; revised March 30, 2021 and April 17, 2021; accepted April 18, 2021. Date of publication May 24, 2021; date of current version January 5, 2022. This work was supported in part by the National Natural Science Foundation of China under Grant 61871460, in part by the Shaanxi Provincial Key Research and Development Program under Grant 2020KW-003 and Grant 2021KWZ-03, and in part by the Ministry of Science and Technology Foundation Funded Project under Grant 2020AAA0106900. Haokui Zhang and Chengrong Gong contributed equally to this work. *(Corresponding author: Ying Li.)*

Haokui Zhang is with the National Engineering Laboratory for Integrated Aerospace-Ground-Ocean Big Data Application Technology, School of Computer Science, Northwestern Polytechnical University, Xi'an 710129, China, and also with Intellifusion, Shenzhen 518000, China (e-mail: hkzhang1991@mail.nwpu.edu.cn; zhang.haokui@intellif.com).

Chengrong Gong, Yunpeng Bai, Zongwen Bai, and Ying Li are with the National Engineering Laboratory for Integrated Aerospace-Ground-Ocean Big Data Application Technology, School of Computer Science, Northwestern Polytechnical University, Xi'an 710129, China (e-mail: hkzhang1991@mail.nwpu.edu.cn; gongchengrong@mail.nwpu.edu.cn; cloudbai@nwpu.edu.cn; ydbzw@yau.edu.cn; lybyp@nwpu.edu.cn).

Digital Object Identifier 10.1109/TGRS.2021.3079123

I. Introduction

HYPERSPECTRAL images (HSIs) are captured via hyperpsectral remote sensors, which collect spatial information and spectral information in hundreds of spectral bands, covering a wide range of wavelengths at the same time. As the products of hyperspectral remote sensors, HSIs have rich spectral and spatial information, which are more effective in distinguishing different land-covers compared with other common remote images. Therefore, HSIs are widely employed in various applications due to their advanced features. Identifying the category of each pixel in HSI is a crucial technique for these applications. However, the rich spectral information in HSI improves distinguishability, on the one hand, and it increases the dimensionality of samples, on the other hand, which may affect the classification performance [1]–[3].

To solve this issue, various feature extraction approaches have been introduced into HSI classification. In the early approaches, most feature extraction approaches used in classification frameworks were handcrafted feature extractors. During this stage, two main frameworks are proposed one after another. The older one is the spectral information-based classification, where only spectral information is exploited. Methods that use only spectral information fail to confront two situations: 1) different objects exhibit similar spectral features; 2) the same objects located in different locations emerge with different spectral features. As a result, these methods generally show relatively poor performance. Latter, both spectral and spatial information were employed to improve classification accuracy. Spectral and spatial information-based HSI classification methods have two main types. The first one uses the spectral and spatial information separately. For instance, the spatial dependence is extracted in advance through spatial filters, such as morphological profiles (MPs) and attribute profiles (APs). Then, spatial dependence information is combined with spectral information. Alternatively, the spatial information can be exploited to refine the classification results obtained by using spectral information only via a regularization process, such as Markov Random Field (MRF) and graph cut. The other one is to combine spectral and spatial information to generate spectral-spatial features, which are then used to predict classification results. For example, several 3-D filters (e.g., 3-D wavelet filters, 3-D Gabor filters [24], and 3-D scattering wavelet filers) have been applied on HSIs to extract spectral-spatial features.

Recently, inspired by the promising performance of deep learning (DL) techniques, various DL models, such as

1558-0644 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:42:04 UTC from IEEE Xplore. Restrictions apply.

stacked autoencoder (SAE), deep belief network (DBN), recurrent neural network (RNN), and convolutional neural network (CNN) have been introduced into HSI classification, leading to significant achievements [4]–[7]. Learning from the experience accumulated in the early stage of HSI classification, most DL-based HSI classification methods combine spectral and spatial information to accomplish the classification task. More specifically, one of the most significant advantages of DL is the ability to extract efficient deep features from raw images directly. This advantage is fully exploited in DL-based classification methods, where DL models are applied to extract deep spectral-spatial features from raw HSI data. For instance, Chen *et al*. [4] applied an unsupervised deep feature learning model SAE to extract spectral and spatial features separately, and then combined the extracted features to generate spectral-spatial features for classification. Later, Chen *et al*. [5] replaced SAE with DBN and proposed a DBN-based spectral-spatial feature extraction framework. Besides these unsupervised DL models, supervised CNN models have been exploited to extract spectral-spatial features. In [6], [8], and [9], 2-D convolutional neural network (2-D-CNN) is developed to extract spectral-spatial features from compressed HSI data. Zhang *et al.* [10] proposed a deep 3-D lightweight convolutional network which consists of tens of 3-D convolution layers. Compared with SAE and DBN, CNN-based HSI classification methods yield better performance and have become the mainstream of DL-based approaches for HSI classification in recent four years.

Most classification approaches are based on manually designed DL models. However, investigating state-of-the-art neural networks is not trivial and it requires substantial efforts, especially in HSI classification, and different HSIs are collected by different sensors with different spatial resolutions, a different number of spectral bands, different spectral ranges, etc. Correspondingly, different HSIs require different models to process, which further increases the workload of designing architectures. In addition, previous work has typically employed a patch-to-pixel framework, where a patch is cropped from the original HSI and all the information of this patch is fed to the trained model to identify a single pixel located at the center of the cropped patch. To obtain the classification result of an entire HSI with a spatial resolution of $M \times N$, a sliding window strategy is adopted to crop $M \times N$ patches. Then, these cropped patches are fed into a trained model for classification. In the patch-to-pixel framework, the overlap regions of different patches are calculated repeatedly, which increases computational cost and time cost. In addition, the classification accuracy is sensitive to the size of cropped patches. Larger patches imply richer contextual information (spatial information). However, larger patches also represent a lower proportion of information in pixels for the central pixel in the patch. The patch size which is too small or too large is not conducive to improve classification accuracy. As a result, it is difficult to discover a proper patch size.

In this article, we attempt to address the aforementioned issues from two angles. First, we propose a 3-D asymmetric neural architecture search (3-D-ANAS) algorithm and leverage it to eliminate the tedious and heuristic procedure of manually design of network architectures from a DL-based pipeline. In the past three years, a growing interest in developing algorithmic solutions to automate the process of architecture design. Architectures found by neural architecture search (NAS) algorithms have achieved highly comparable performance in both high-level vision tasks [11], [12], and low-level vision tasks [13]–[15]. Very recently, Chen *et al*. [16] proposed automatic 1-D Auto-CNN and 3-D Auto-CNN for HSI classification and achieved encouraging performance. Compared with manually designed architectures, the architectures of 1-D Auto-CNN and 3-D Auto-CNN outperformed with fewer parameters. However, in [16], like most of the other NAS works, only topological structures are designed via NAS algorithms. The widths (the numbers of channels of convolution layers) of different layers of the network are manually set. In addition, the search space built in [16] still has room for improvement. In this work, we build a hierarchical search space including an inner search space and outer search space, which are in charge of designing cell architectures and deciding cell widths, respectively. For the inner search space, we build a 3-D asymmetric decomposition search space. For the outer search space, we employ cell sharing strategy to save memory resources. Second, we adopt a pixel-to-pixel classification framework to improve the inference efficiency. Within the pixel-to-pixel classification framework, the trained model can take patches with arbitrary size as input and identify all pixels of the input patch at one time, which significantly enhanced the inference speed.

In summary, the work of this article focuses on automatically searching for 3-D asymmetric networks for fast HSI classification. Our main contributions are summarized as follows:

1) Based on the gradient search strategy, we propose a 3-D asymmetric NAS algorithm, termed 3-D-ANAS. The proposed 3-D-ANAS has a hierarchical search space which includes inner search space and outer search space. 3-D-ANAS is capable of searching for both topological structure and network widths. By analyzing the characteristics of HSIs, we establish a 3-D asymmetric decomposition search space as inner search space. To save memory, we employ a cell-sharing strategy in outer search space.
2) In order to overcome the duplicate operation problem in the patch-to-pixel classification framework employed in previous works, we propose a pixel-to-pixel classification framework, which increases the inference speed to $10\times$ of 3-D Auto-CNN (which represents the state-of-the-art NAS-based HSI classification method).
3) For the proposed pixel-to-pixel classification framework, we propose two augmentation inference strategies, which further improve the classification performance. Experimental results on three HSI datasets captured by different sensors after 2001 demonstrated that architectures found by our proposed 3-D-ANAS outperform other manually designed 3-D CNN-based HSI classification methods and NAS-based HSI classification methods in classification accuracy while having much faster inference speed.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:42:04 UTC from IEEE Xplore. Restrictions apply.

The rest of this article is organized as follows. Section II presents the related work, consisting of DL-based HSI classification methods and NAS algorithms. Section III elaborates on our proposed method, including inner topological architecture search, outer widths search, pixel-to-pixel classification framework, and training strategy. In Section IV, we first describe datasets and experimental setups; then we compare and analyze the experimental results; finally, we conclude this article and point out our future work in Section V.

II. Related Work

A. DL for HSI Classification

Broadly speaking, there are four basic types of DL models, including SAE, DBN, CNN, and RNN. Since 2013, when Lin *et al*. [17] proposed to use SAE to extract spectral-spatial features and achieved impressive classification performance, an increasing number of DL-based HSI classification methods have been investigated. Until now, all four types of DL models and corresponding derivative models have been introduced to HSI classification.

Earlier work exploited SAE and DBN to extract deep features for HSI classification. To our knowledge, the earliest attempt can be found in [17], where Lin *et al*. proposed to use SAE to extract spectral-spatial features from preprocessed HSIs. The preprocessing operation is spectral compression, where principal component analysis (PCA) [18] is employed to reduce the data dimensionality by compressing the raw HSI containing hundreds of spectral bands into a compressed HSI which consists of tens of spectral bands. Based on this work [17], Lin *et al*. proposed a new SAE-based HSI classification framework, where both compressed HSI and raw HSI can be utilized [4]. Specifically, the features extracted via the framework proposed in [17] are regarded as spatial features, while the 1-D features extracted from raw HSI with SAE are regarded as spectral features. Once spatial features and spectral features are extracted, they are combined to generate spectral-spatial features. Liu *et al*. replaced SAE with DBN in the framework proposed in [4] and proposed a DBN-based spectral-spatial HSI classification method [5]. Ma *et al*. [19] exploited SAE to learn the effective features, where a relative distance prior is added into the subsequent fine-tuning process. In the above four methods, spectral information does not require any preprocessing, but spatial information must be flattened to a 1-D vector, as SAE and DBN only accept 1-D input. However, the flattened features do not retain the same spatial information that the original image may contain.

In the last five years, various CNN models have been employed to classify HSIs. CNN-based HSI classification methods were proposed and compared with SAE and DBN. Statistically, the number of methods using CNNs for HSI classification has grown fastest and the performance of CNNs is generally superior. After 2016, CNN-based methods have become the mainstream of DL-based HSI classification approaches. Generally speaking, CNNs contain three main types, including 1-D-CNN, 2-D CNN (2-D-CNN), and 3-D-CNN. In the 1-D-CNN-based work, convolution layers implement convolution operations on the input samples along the spectral dimension. For instance, Hu *et al*. [59] built a 1-D-CNN network containing one convolution layer, one polling layer, and two fully connected layers for HSI classification. Zhang and Li [20] proposed to use a 1-D-CNN to extract spectral-spatial features from 3×3 pixels. Meanwhile, Mei *et al*. exploited a similar 1-D-CNN to classify HSIs in [21]. In 2-D-CNN-based classification methods, 2-D-CNN models and dimension reduction algorithms are always used jointly. Specifically, HSIs are always compressed via a dimensional reduction algorithm, and then convolved with 2-D kernels. For example, in [22], R-PCA is used to condense the entire HSI, then 2-D CNN is employed to extract spatial features. Later, a similar framework was proposed in [23], where the top three principal components were extracted from the raw HSI by using PCA, then fed into 2-D-CNN to extract spatial features. With 3-D-CNNs, robust spectral-spatial features can be extracted from raw HSIs directly, as both the model and the data have a 3-D structure. In fact, compared with 1-D-CNN and 2-D-CNN-based methods, 3-D-CNN-based methods generally have higher classification accuracy. In [24] and [7], 3-D-CNNs were adopted to extract spectral-spatial features from HSI data without any preprocessing. Licciardi *et al.* [8] employed spectral and spatial residual blocks consecutively to learn spectral and spatial representations separately. Very recently, Zhang *et al*. [10] proposed a deep 3-D lightweight convolutional network and combined it with transfer learning to perform HSI classification in the condition of limited training samples. RNN is mainly designed to handle sequential data. Compared with SAE, DBN, and CNN, approaches based on RNNs are relatively few. Wu and Prasad [25] proposed a convolutional recurrent neural network (CRNN) for HSI classification, consisting of a few convolution layers followed by recurrent layers.

Broadly speaking, 3-D-CNNs yield superior performance in HSI classification compared with SAE, DBN, and RNN. However, all the 3-D-CNN-based HSI classification methods introduced above are still based on manually designed architectures.

B. Neural Architecture Search

Recently, NAS algorithms have attracted widespread attention and outperformed manually designed architectures on various computer vision tasks, including image classification [26], semantic segmentation [11], object detection [27], image denoising [13], [14], [28], image super-resolution [29], designing data augmentation strategy [30], and designing activation function [31]. Differing from conventional DL-based image processing work, where architectures of networks are designed manually, NAS aims to design automated approaches to discover high-performance neural architectures. Until now, there are three mainstream search strategies, including evolutionary algorithms (EAs), reinforcement learning (RL), and gradient-based architecture search. RL-based works, in [26] and [32], RL techniques policy gradients and Q-learning are, respectively, employed to train an RNN that works as meta-controller to generate potential architectures. References [29], [33], [34] evolve a set of stochastic

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:42:04 UTC from IEEE Xplore. Restrictions apply.

construction models by EA to obtain better architecture. EA and RL-based methods are normally inefficient in search, often requiring a large number of computations. To remedy this deficiency, multiple speed-up techniques are proposed. For instance, hypernetworks [35], network morphism [36] and shared weights [37]. Gradient-based NAS methods were proposed in the last two years. Compared with the previous two types, gradient-based NAS methods have been developed for a shorter time. The earliest attempt can be found in [12], where differentiable architecture search (DARTS) was proposed for the first time. Differing from EA and RL-based methods which train a huge number of student networks, DARTS just train a single supernet in the search phase, enabling the searching stage to be quite efficient. Following DARTS, various gradient-based NAS methods have been proposed for image classification [38], semantic segmentation [11], image restoration [13], [15], and HSI classification [16].

Besides the three mainstream search strategies, Bayesian optimization (BO) and graph model are also widely used in NAS. These BO and graph model adopted methods also demonstrate promising performance. For instance, Chen *et al.* [39] proposed to use BO-based optimization service to search for architectures for dense prediction cells, which can be used on the tasks scene parsing, person-part segmentation, and semantic image segmentation [40]. Kandasamy *et al.* [41] developed a BO framework and a pseudo distance metric for optimizing functions on neural networks architectures. Except for gradient-based NAS methods, other NAS methods all face the problem of sampling new candidate networks. Wang *et al.* [42] proposed to sample new candidates according to BO criterion, which samples from regions where good quality candidate networks were found before. Graph-based schemes are generally used in predictor-based NAS methods. For example, in [43], Ning *et al.* modeled the information flow of data processing in a directed acyclic graph (DAG). Zhang *et al.* [44] trained a graph hypernetwork to predict the parameters of random sampled architectures. Finally, the predict parameters are used to evaluate the fitness of these sampled architectures.

Our proposed 3-D-ANAS is most closely related to automatic 3-D Auto-CNN [16]. Both are gradient-based NAS methods and are proposed for HSI classification. Automatic 3-D Auto-CNN builds a 2-D convolution search space which is similar to that of DARTS. In addition, similar to previous 2-D CNN-based HSI classification methods, automatic 3-D Auto-CNN has a preprocessing stage, in which raw HSI is condensed along the spectral dimension via a point-wise convolution. Compared with automatic 3-D Auto-CNN, our proposed 3-D-ANAS has four main differences: 1) 3-D asymmetric decomposition inner search space. By analyzing the characteristics of HSI, we build a 3-D asymmetric decomposition inner search space, where 3-D asymmetric decomposition convolution operations are employed. Since both operations in the search space and HSI data are 3-D structures, no preprocessing operations are required in our framework; 2) search for different architectures for different layers, increasing flexibility; 3) hierarchical search space. We build a hierarchical search space that can search both topological architecture and network width; and 4) a pixel-to-pixel classification framework. To overcome the problem of repetitive operation, we propose a pixel-to-pixel classification framework, which significantly improves the inference speed.

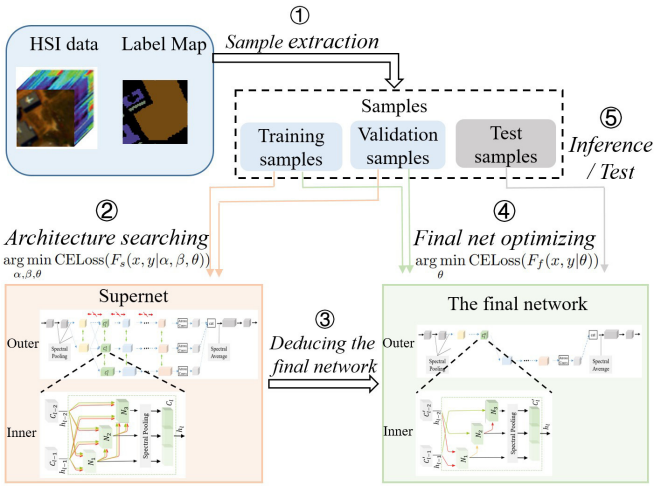


Fig. 1. Work flow of the proposed 3-D-ANAS.

III. Proposed Method

A general workflow of 3-D-ANAS is shown in Fig. 1. As indicated previously, our proposed 3-D-ANAS has a hierarchical search space that consists of an inner search space and an outer search space.

By analyzing the characteristics of HSIs, we build a 3-D asymmetric decomposition search space as the inner search space, which is used to search for topological architectures. The outer search space contains three candidate paths one for each of the widths. The outer search space is employed to decide the widths for different layers. In addition, the classification framework used in this article is a pixel-to-pixel classification framework.

In this section, we first present our 3-D asymmetric decomposition search space. Then we explain how to search for widths for different layers in the outer search space. Finally, we introduce the pixel-to-pixel classification framework and our training strategy.

A. Inner Topological Architecture Search

1) 3-D-Asym-d Search Space: Different from previous works [12], [13], [16], which search for a single computation cell as the basic block and then build the whole network by stacking the found block with different widths, our proposed 3-D-ANAS does not share architecture across different layers, it searches for L different cells for the final network which contains L layers and then builds the final network by stacking the L cells in order. To be specific, for each layer, we build three supercells, which have different widths and each contains a sequence of N nodes. Each node is connected to two inputs of the current supercell and nodes before it processes through different paths. In the supercell, each path has all candidate

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:42:04 UTC from IEEE Xplore. Restrictions apply.

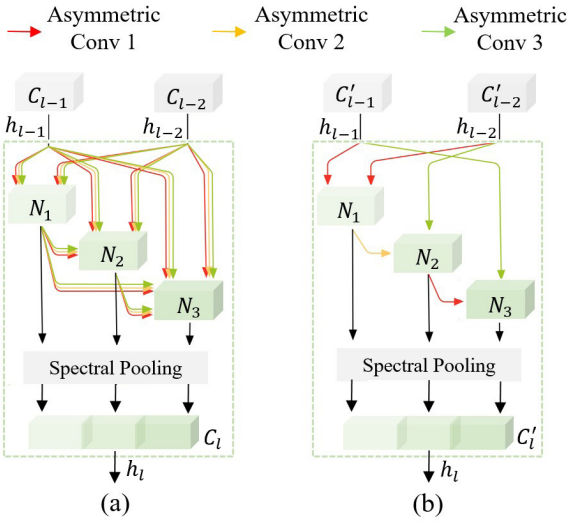


Fig. 2. 3-D-asym-d search space and topological architecture search. Left: 3-D-asym-d search space. For each layer, we build a supercell that consists of all possible operation types, which are represented as arrows with different colors. Right: the result of topological architecture search, a compact cell, where each node only keeps the two top valuable inputs, each of which is fed to the selected operation. Here, we only show three nodes and three asymmetric convolution operations for clear exposition. (a)–(c) Conv 1–3 denote different convolution operations.

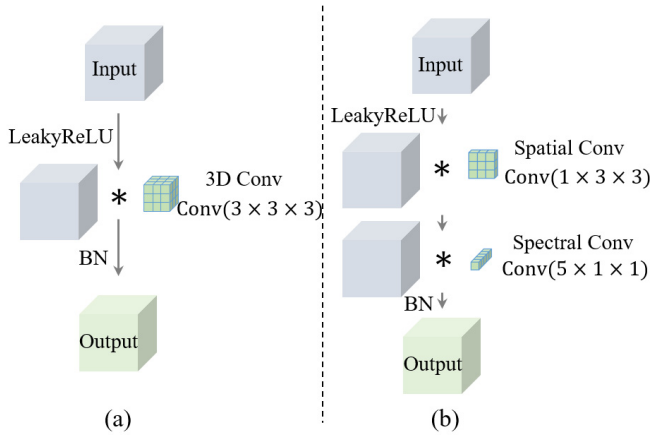


Fig. 3. 3-D convolution and 3-D asymmetric decomposition convolution. $*$ represents convolution operation. (a) 3-D convolution. (b) 3-D asymmetric decomposition convolution. In 3-D asymmetric convolution, the spatial and spectral information is processed separately. The kernel sizes along spatial dimension and spectral dimension can be different (asymmetric).

operations, each of which corresponds to a learnable weight α. The task of inner architecture search is learning a set of path weights α, which can make the supercell achieve the best performance. During searching, the path weights are optimized via the gradient descent algorithm. After searching, a compact cell can be obtained by only keeping the top two valuable paths for each node.

An illustration of the supercell built for each layer is shown in Fig. 2(a) and the corresponding found compact cell is shown in Fig. 2(b). In Fig. 2, one supercell in layer l is represented as C_l. C_l takes the outputs of the previous cell C_{l-1} and the cell before previous cell C_{l-2} as inputs and outputs h_l. C_l consists of a sequence of N nodes (N is set to three for clear exposition in Fig. 2). Inside C_l, node i takes the two inputs of current cell and the outputs of all previous nodes as inputs. The output of the ith node is

$$x_{l,i} = \sum_{x_j \in I_{l,i}} O_l^{j\to i}(x_j)$$
$$O_l^{j\to i}(x_j) = \sum_{k=1}^{S} \alpha_l^{k,j\to i} o^k(x_j) \tag{1}$$

where $x_{l,i}$ is the output of the ith node in layer l. $I_{l,i} = \{h_{l-1}, h_{l-2}, x_{l,j<i}\}$ denotes the input set of the ith node. h_{l-1} and h_{l-2} are the outputs of cells C_{l-1} and C_{l-2}. $O_l^{j\to i}$ represents the set of possible layer types. $\alpha_l^{k,j\to i}$ is the weight of operator o^k, which is related to l, as we search for different cell architectures for different layers. The cells in the same layer share the same set of continuous variables α. $\{o^1, o^2, \ldots, o^S\}$ correspond to S possible operations. The output of C_l is denoted as h_l, which is the concatenation of the outputs of all nodes in C_l and it can be expressed as

$$h_l = \text{Cell}(h_{l-1}, h_{l-2})$$
$$= \text{Concat}\{x_{l,i} | i \in \{1, 2, \ldots, N\}\}. \tag{2}$$

2) 3-D Asymmetric Decomposition Convolution: In this article, we provide eight different operations, including three common convolution operations, three separable convolution operations, skip connection operation, and discarding operation. The eight operations are shown as follows:

1) *Con_3-3:* LReLU-Conv($1 \times 3 \times 3$)-Conv($3 \times 1 \times 1$)-BN;
2) *Con_5-3:* LReLU-Conv($1 \times 5 \times 5$)-Conv($3 \times 1 \times 1$)-BN;
3) *Con_3-5:* LReLU-Conv($1 \times 3 \times 3$)-Conv($5 \times 1 \times 1$)-BN;
4) *Sep_3-3:* LReLU-Sep($1 \times 3 \times 3$)-Sep($3 \times 1 \times 1$)-BN;
5) *Sep_5-3:* LReLU-Sep($1 \times 5 \times 5$)-Sep($3 \times 1 \times 1$)-BN;
6) *Sep_3-5:* LReLU-Sep($1 \times 3 \times 3$)-Sep($5 \times 1 \times 1$)-BN;
7) *Skip_Connection:* $f(x) = x$;
8) *Discarding:* $f(x) = 0$.

where leaky rectified linear unit (LReLU), batch normalization (BN), Conv, and Sep represent LeakyReLU activation function [45], BN, common convolution, and separable convolution. Compared with ReLU which toughly sets all negative values to zeros, LeakyReLU has a small slope for negative values and reserves small, nonzero gradients for these negative values, which is beneficial for avoiding dying ReLU problem. So, we adopt LeakyReLU as an activation function in the proposed method. Note that, in this article, each convolution operation is decomposed into a sequence of two decomposition convolution layers for the spatial dimension and spectral dimension. Fig. 3 shows an instance, where a 3-D convolution with kernel size of $5 \times 3 \times 3$ is decomposed into two convolutions with kernel sizes of $1 \times 3 \times 3$ and $5 \times 1 \times 1$, respectively. Here, we adopt asymmetric decomposition convolution for the following two main reasons: 1) Compared with conventional 3-D convolutions, the 3-D asymmetric decomposition convolution has a deeper structure and fewer parameters, while having the same receptive field (Receptive field denotes the region of input space that affects a particular unit of the network [46]).

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:42:04 UTC from IEEE Xplore. Restrictions apply.

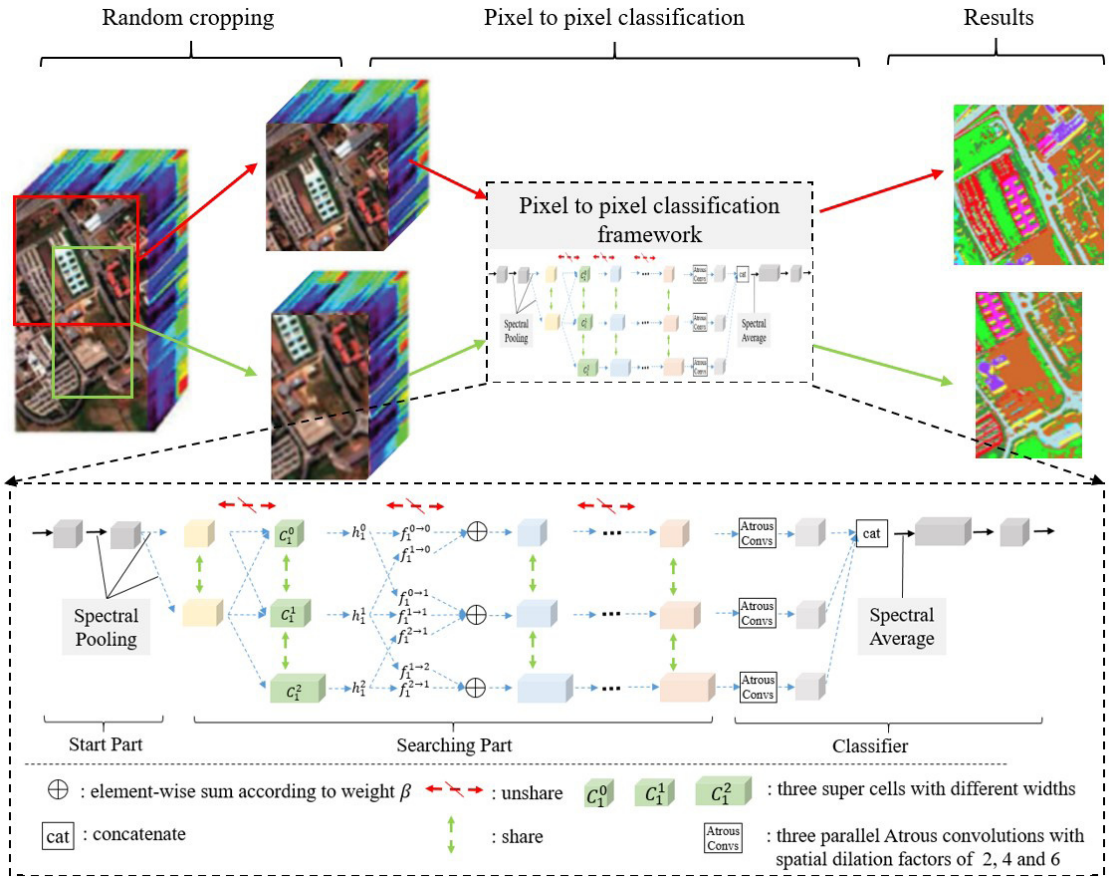


Fig. 4. Proposed framework. Upper half: pixel-to-pixel classification framework, where patches with arbitrary size is cropped from raw HSI. Then these cropped patches are fed into network to get corresponding classification results. Lower half: the supernet that consists of start part, searching part and classifier part. During searching, the searching part provides three candidate paths with different widths for each layer. After searching, just on path in each layer is retained according to optimized path weight β.

The advantages of a 3-D asymmetric decomposition convolution have been fully demonstrated in [47]–[49] and 2) by in-depth analysis of the architectures of HSIs, we find that HSIs have a relatively low spatial resolution and an extremely high spectral resolution. The sizes of kernels along the spatial dimension and spectral dimension should be different. However, increasing the kernel size along the spatial dimension or spectra dimension in the common 3-D convolution will bring too many parameters, which may affect the final classification performance. By decomposing the common 3-D convolution into a pseudo 3-D convolution along the spatial dimension and a pseudo 3-D convolution along the spectral dimension, it is able to increase the kernel size along the spatial dimension or spectral dimension to get a 3-D asymmetric convolution without introducing too many parameters. For instance, a $3 \times 3 \times 3$ sized 3-D kernel has 27 parameters. Increasing the kernel size along the spectral dimension to 5, the number of parameters will increase to $3 \times 5 = 45$. The corresponding 3-D asymmetric decomposition version has only $3 \times 3 + 5 = 14$ parameters, which is less than one-third of the common 3-D version.

B. Outer Widths Search

The final architecture is constructed by stacking the found L cells with different widths. Once the topological architectures of the L cells are found via inner architecture search, we need to either heuristically set or automatically search for width for each cell to build the final network. In conventional CNNs, the change of widths is always related to the change of spatial resolutions. For example, doubling the widths of the following convolution layers after the spatial resolution of features is reduced. However, to preserve pixel-level information for pixel-to-pixel classification, we do not employ any operations which may reduce the spatial resolution. Thus, the conventional experience of setting the width no longer applies to our case. Aiming to solve this issue, we build an outer search space to automatically search for widths and accomplish the task of setting widths via a NAS algorithm.

1) Outer Search Space: The outer search space we built for searching for the widths consists of three candidate paths corresponding to different widths, as shown in the bottom half of Fig. 4. The complete network has three main parts, including the start part, the searching part, and the classifier part. Next, we elaborate on these three parts:

1) Start part, two common 3-D convolution layers, each of which is followed by a BN layer. In each convolution layer, extracted features are compressed via pooling operation along the spectral dimension. In the start part, both 3-D convolution layers have 32 channels (width is 32). The kernel size is $(5, 3, 3)$, where the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:42:04 UTC from IEEE Xplore. Restrictions apply.

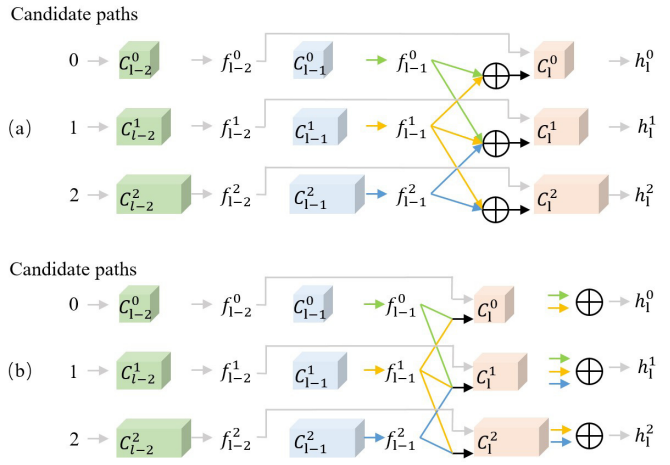


Fig. 5. Cell-sharing strategy. (a) Using cell sharing. Cell C_l^1 is shared by features from different candidate paths. During the search, C_l^1 is called once to process all features from three candidate paths. (b) Without using cell sharing. During the search, C_l^1 is called three times to process features from three candidate path, separately.

first dimension corresponds to the spectral dimension and the last two dimensions are spatial dimension. The corresponding stride is set to (2, 1, 1).

2) Searching part, including L layers and each layer consisting of three supercells of different widths. This part represents the outer search space. In several classical networks, such as Inception V1 to V4 and MobileNet V3, the topological architecture of different stages is different. In fact, some of the early NAS work adopt sequential search spaces and search for different structures for different layers [50]. But, most current NAS work replaces the sequential search space with cell-based search space deal to that searching in sequential search space has a huge computational cost [26]. Cell-based search space just contains two types of cells. The final network is built by stacking the basic cells repeatedly. Although the final network contains tens of cells, the cells in different layers are the same. Cell-based search space reduces computation cost, on one hand, and, on the other hand, it reduces the flexibility of search space. Our 3-D-ANAS also adopts cell-based search space, but it searches for L different cells for a network consisting of L layers. Each layer contains several cells of different widths, which share the same architecture parameters α (formula (1)). Compared with other cell-based search space methods, the search space of our 3-D-ANAS is more flexible.

3) Classifier, which compresses the outputs of the last layer of the searching part to the same dimension then concatenate the compressed features to generate the concatenated feature. Once the concatenated feature is generated, it is averaged over the spectral dimension. Finally, the averaged feature is fed into a sequence of two 2-D convolution layers to get the final classification result.

As shown in the lower part of Fig. 4, in the searching part, each layer has three supercells except the first layer which contains two supercells. Three cells at layer l are C_l^0, C_l^1 and C_l^2, widths of which are $\gamma^0 \times W$, $\gamma^1 \times W$ and $\gamma^2 \times W$, where W denotes the basic width, which is set to 8 during searching and 16 in the final found network. γ^0, γ^1 and γ^2 are the width variation factors, which are set to 1.0, 1.5, and 2.0 in this article. The output of each layer can be expressed as

$$h_l = h_l^0, h_l^1, h_l^2 \tag{3}$$

where h_l^i represents the output of cell C_l^i. The width of it is $\gamma^i WN$, where N denotes the number of nodes in cell C_l^i.

2) Cell Sharing: Supercells decide search spaces of different layers. The search space of the whole network is made up of a sequence of supercells, and it is called as supernet. The supernet contains all possible layer types and corresponding controlling parameters (α and β). In supernet, each layer has three candidate paths. During searching, in layer l, the ith cell C_l^i also has two inputs: 1) one is C_{l-2}^i, the ith cell in layer $l-2$ and 2) the other approach is chosen from outputs of the previous layers, C_{l-1}^{i-1}, C_{l-1}^i, and C_{l-1}^{i+1}. As the widths of different cells in the same layers are different, 3-D point wise convolution is adopted to map transform outputs h_l^{i-1}, h_l^i and h_l^{i-1} to f_l^{i-1}, f_l^i and f_l^{i+1} to match the requirement of cell C_l^i. f_l^{i-1}, f_l^i and f_l^{i+1} have same width, that is $\gamma^i W$. The output of the cell C_l^i is computed with

$$h_l^i = \sum_{k=i-1}^{i+1} \beta_l^{k,i} C_l^k \left(f_{l-1}^k, f_{l-2}^i\right) \tag{4}$$

where $\beta_l^{k,i}$ is the learnable weight of candidate path k. During searching, the learnable weights of different candidate paths are updated via gradient descent algorithm, similar with what we do to learn operation weights α. From formula 4, we can see that cell C_l^i is called three times. In other words, three portions of memory are occupied. Therefore, we propose a cell sharing strategy to save memory and search efficiency. With the proposed strategy, formula 4 is rewritten as

$$h_l^i = C_l^i \left(\sum_{k=i-1}^{i+1} \beta_l^{k,i} f_{l-1}^k, f_{l-2}^i \right) \tag{5}$$

where cell C_l^i is called once. By using the cell-sharing strategy, the memory consumption is reduced by 1/3. Fig. 5 shows the comparison between using cell sharing and not using cell sharing. With cell sharing, we first combine the outputs of cell C_{l-1} according to corresponding weights then call C_l^i to process the combined output in a single call. Alternatively, cell C_l^i is shared by three candidate paths during the search. The theory behind our proposed cell sharing strategy is similar with meta kernel [51], where different kernels are summed according to weights then fed into the convolution operation to save memory. Cell sharing can improve our proposed 3-D-ANAS in two aspects: 1) cell sharing reduces the demand on computing resources, enabling our 3-D-ANAS to be applied to a wider range of domains and 2) cell sharing improves the searching efficiency. As it saves memory consumption, we can use larger batch sizes during the search to increase the search speed. We can also build a deeper and wider supernet to achieve higher accuracy.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:42:04 UTC from IEEE Xplore. Restrictions apply.

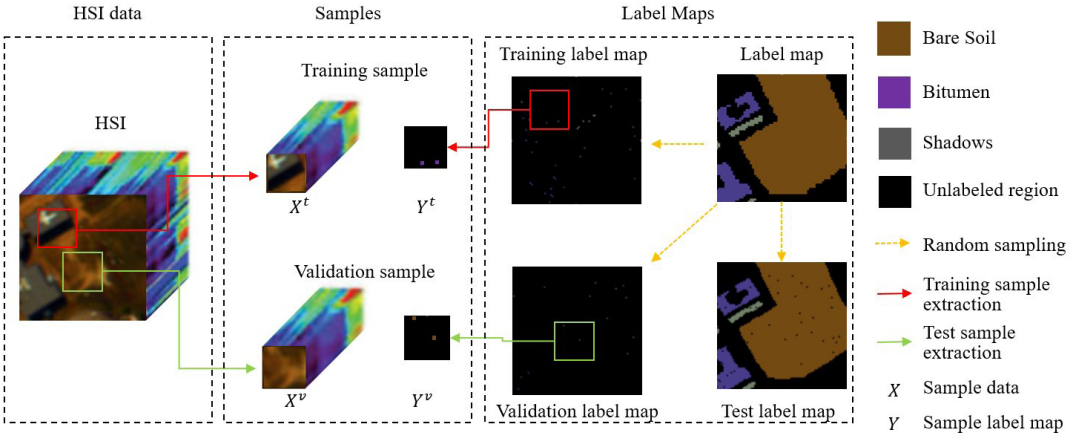


Fig. 6. Illustration of sample extraction. Label map is randomly divided into three submaps, including training label map, validation label map, and test label map. To generate a training sample, we first randomly crop a $W_1 \times W_2$-sized patch with sparse labeling information as the label of the training sample (Y^t). Then, according to the spatial location of Y^t, we correspondingly crop a $W_1 \times W_2 \times L$-sized 3-D cuboid from HSI as the data of training sample (X^t). We extract validation samples in the same way except that label patches are extracted from validation label map.

Theoretically, the outer widths search is an extension of the inner architecture search. However, the way to select candidate paths of different widths is different from the way to get the final unit architecture. Here, different layers cannot be considered independently. If we just retain the candidate path regarding the maximum path weight β in each layer, then the widths of adjacent layers in the final network may change drastically. For instance, layer $l-1$ chooses cell C_{l-1}^0 and layer l chooses cell C_l^2. According to [52], this kind of drastic width change has a negative impact on the efficiency. In addition, in our supernet, there is no path between C_{l-1}^0 and C_l^2, as illustrated in Fig. 5. Building a new path that does not exist in the search space is unreasonable. To overcome this issue, we regard the ?-values as probabilities, then use the Viterbi decoding algorithm [53] to choose a candidate path to obtain the final widths search result.

C. Pixel-to-Pixel Classification Framework

Previous spectral-spatial HSI classification approaches usually adopt patch-to-pixel classification framework, where a patch cropped from HSI is fed into trained networks to get the class of a single pixel located in the center of the cropped patch. In short, all information in a patch is used to classify a single pixel (patch-to-pixel). Two main drawbacks of patch-to-pixel classification framework are: 1) low computing efficiency. The overlap regions of patches cropped for adjacent pixels are repeatedly computed and 2) sensitivity to patch size. The classification accuracy is sensitive to the patch size. Larger patch indicates richer contextual information. However, larger patches also represent that the information proportion of the center pixel among pixels in the patch is lower. In addition, different model structures and different HSI datasets have different practical sizes. For instance, the practical patch size is 5 for 3-D-CNN model proposed in [7], while it is 27 for 3-D-CNN model proposed in [24]. For spectral-spatial residual network (SSRN) [8], the practical patch size is 7. Generally, to obtain the practical patch size, abundant inquiry experiments are carried out.

In this article, we adopt a pixel-to-pixel classification framework. Similar to the patch-to-pixel framework, the pixel-to-pixel framework also takes patches as input. Different from the patch-to-pixel framework, pixel-to-pixel framework classifies all pixels in the input patch. In addition, our pixel-to-pixel classification framework is able to process a patch of any size. Our pixel-to-pixel framework adopts structures adopted in segmentation tasks. However, HSI classification has higher requirements in pixel-level accuracy compared with segmentation tasks. So, our pixel-to-pixel framework discards any spatial resolution reduction operation to preserve pixel-level information. In our pixel-to-pixel framework, the spatial resolution remains unchanged from the input layer to the final prediction result. Meanwhile, to reduce the number of computations, spectral pooling is employed to condense features along the spectral dimension.

D. Training Strategy

As the basic framework, we used in this article is a pixel-to-pixel classification framework, which is different from the patch-to-pixel classification framework used in previous works. The training samples used in this article are also different from previous work. Next, we explain how to extract samples and train our pixel-to-pixel framework. A labeled sample contains two parts, input sample data X and corresponding label Y. In the patch-to-pixel framework, X is a 3-D cuboid. For instance, a $W \times W \times L$-sized cuboid, where W is the patch size and L denotes the number of spectral bands. The corresponding label Y is a single integer. In our pixel-to-pixel classification framework, input data X is still a 3-D cuboid. For example, a $W_1 \times W_2 \times L$-sized cube, where $W_1 \times W_2$ are spatial resolution and W_1 and W_2 can be different. The corresponding label Y is a $W_1 \times W_2$-sized 2-D label map. In HSI datasets, not every pixel is labeled, so the label map Y is sparse. The procedure of sample extraction is shown in Fig. 6.

Our proposed 3-D-ANAS has two optimizing stages, including 1) architecture searching stage (supernet optimizing stage) and 2) the optimizing stage of the final network.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:42:04 UTC from IEEE Xplore. Restrictions apply.

In the searching stage, we randomly generated 400 training samples. This training set size was obtained via repeated investigation experiments. Specifically, after the training label map was generated. We tested to generate [100, 200, 300, 400, 500, 600] training samples for the searching stage; then, we found that supernet cannot get a good convergence curve when the training sample size is too small but it shows a good convergence curve when the training sample size is larger than 400 (training supernet with 400, 500, and 600 samples show similar convergence curve). So, we infer that generating 400 training samples is large enough to train the Supernet.

Then we randomly distributed training samples into two subsets Train_θ and Train_A, each of which has 200 training samples. Train_θ is used to optimize supernet parameters θ, which are learnable parameters in convolution layers and BN layers. Train_A is used to update the architecture parameters α and β. The optimization function of searching process can be expressed as

$$\arg\min_{\alpha,\beta,\theta} \ \text{CELoss}(F_s(x, y|\alpha, \beta, \theta)) \tag{6}$$

where CEloss is cross entropy loss, F_s denotes supernet, x and y are the sample data and the sample label. During searching stage, architecture parameters (α and β) and supernet parameters (θ, the learnable kernels of different convolution layers and the learnable parameters of BN layers in supernet.) are alternately updated. Specifically, the update formula of architecture parameters is

$$\begin{aligned} m_t &= \gamma_1 \cdot m_{t-1} + (1-\gamma_1) \cdot \nabla_{\eta_t} \\ v_t &= \gamma_2 \cdot v_{t-1} + (1-\gamma_2) \cdot \nabla_{\eta_t}{}^2 \\ \hat{m}_t &= \frac{m_t}{1-\gamma_1^t} \\ \hat{v}_t &= \frac{v_t}{1-\gamma_2^t} \\ \eta_{t+1} &= \eta_t - \text{lr} \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon} - \text{lr} \cdot \omega \cdot \eta_t \end{aligned} \tag{7}$$

where $\epsilon = 1e-8$, $\gamma_1 = 0.9$, $\gamma_2 = 0.9999$, m_t and v_t are the first and second moment vectors [54]. ∇_{η_t}, ∇_{n_t} and ω represent gradient, learning rate and weight decay, respectively. η is architecture parameter α or β. The supernet parameters are updated according to

$$\begin{aligned} s_{t+1} &= \mu \cdot s_t - \text{lr} \cdot \omega \cdot \theta_t - \text{lr} \cdot \nabla_{\theta_t} \\ \theta_{t+1} &= \theta_t + s_{t+1} \end{aligned} \tag{8}$$

where μ is momentum term. To evaluate the performance of supernet, we build a validation set Val. Specifically, first, based on the sliding window algorithm, we go through the validation label map and crop a sequence of patches which are adjacent to each other; then we discard these patches which have no labeled pixels and reserve the rest as validation set. We evaluate the supernet for every epoch and retain the best one. At the end of searching stage, we deduce the final network architecture according to the retained supernet.

In the final network optimizing stage, we randomly generate training samples in each iteration and evaluate the found network for every 100 iterations. We stop the optimization process when the classification accuracy on validation set no longer increase and the loss on validation set no longer decreases. The optimization function of this stage is

$$\arg\min_{\theta} \ \text{CELoss}(F_f(x, y|\theta)) \tag{9}$$

where F_f is the final network.

IV. Experimental Results

A. Data Description and Experiment Design

In this section, we organize three groups of experiments. First, to evaluate the performance of our proposed 3-D-ANAS, we carry out experiments on three different HSI datasets which are collected after 2001 and compare the proposed 3-D-ANAS with other 3-D-CNN-based and NAS-based methods. Second, to better comprehend the architectures found by 3-D-ANAS, we conduct architecture analysis experiments. Finally, we present an ablation study to analyze the effectiveness of each component in 3-D-ANAS and the inference speed of architectures found by 3-D-ANAS.

1) Data Description: The three HSI datasets used in this article are Pavia University, Pavia Center, and Houston University. The first two datasets were collected by the Reflective Optics System Imaging Spectrometer (ROSIS) sensor in 2001, during a flight campaign over Pavia, northern Italy. After correction, Paiva University has 103 spectral bands and Paiva Center retains 102 spectral bands. Pavia University covers 610×340 pixels and Pavia Center consists of 1096×715 pixels. Both Pavia University and Pavia Center have nine classes of ground objects. The geometric resolution of these two datasets is 1.3 m. The last dataset Houston University was captured by ITRES-CASI 1500 hyperspectral Imager in June 2012. It covers the regions over the University of Houston campus and the neighboring urban area. Houston University has 144 spectral bands, covering the range from 0.36 to 1.05 μm. It consists of 349×1905 pixels with a high spatial resolution of 2.5 m. 15 land-cover classes are included in this dataset. Sample distribution information is listed in Table I and shown in Fig. 7.

2) Implementation Details: As we discussed in Section III-D, the proposed 3-D-ANAS has two optimizing phases, including the architecture searching phase and the final network optimizing phase. In this section, we first expound implementation details of these two stages in detail, then we introduce experiment design.

3) Architecture Searching: For three datasets employed in this article, we build three supernets, each of which has two layers of supercells. Each supercell contains three nodes. The task of architecture searching is optimizing supernets to get optimal weights of different operations and candidate paths. We crop patches with a spatial resolution of 32×32 as training and validation samples. We train each supernet on a single GTX1080Ti GPU, which has 11 G memory. For Pavia University, Pavia Center, and Houston University, we set the batch size to 5, 5, and 4, respectively. We use Adam optimizer [54] to optimize architecture parameters (α and β). We adopt the default learning rate (0.001) and set weight

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:42:04 UTC from IEEE Xplore. Restrictions apply.

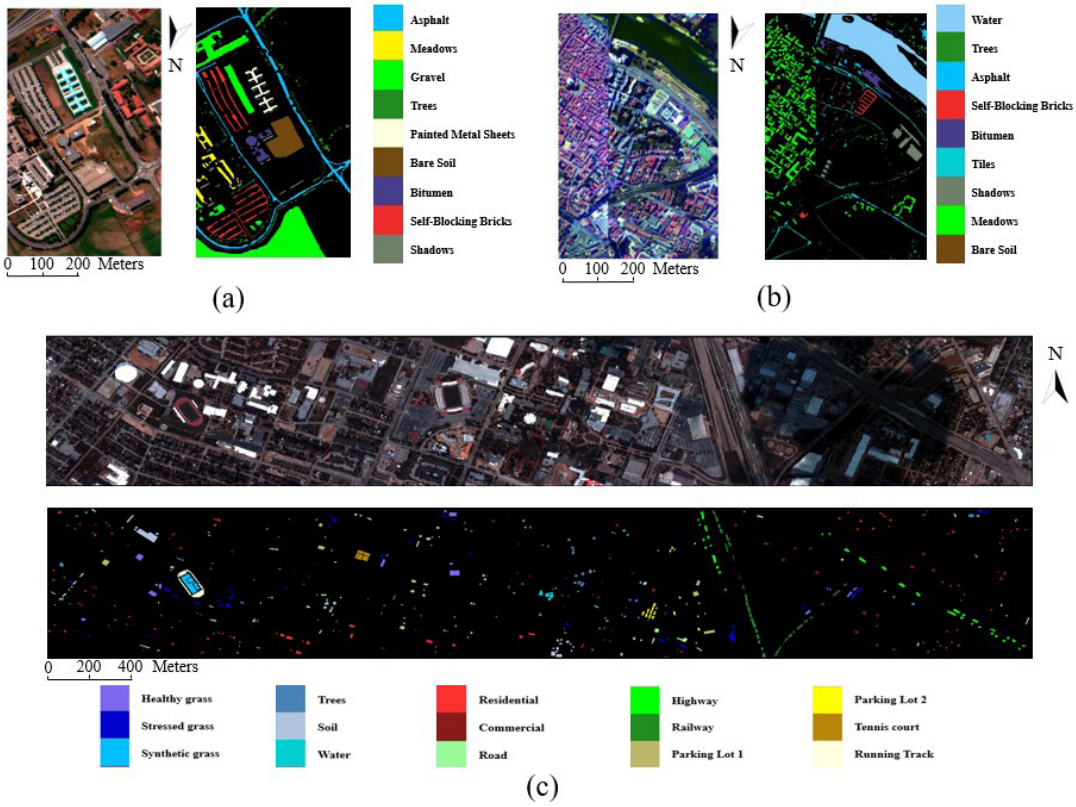


Fig. 7. False color composites and ground truth maps of datasets. (a) Pavia university. (b) Pavia center. (c) Houston university.

TABLE I
SAMPLE DISTRIBUTION INFORMATION OF DATASETS

	Pavia University		Pavia Centre		Houston University	
No.	Category	Labelled samples	Category	Labelled samples	Category	Labelled samples
1	Asphalt	6631	Water	824	Healthy Grass	1251
2	Meadows	18649	Trees	820	Stressed Grass	1254
3	Gravel	2099	Asphalt	816	Synthetic Grass	697
4	Trees	3064	Self-Blocking Bricks	808	Trees	1244
5	Painted Metal Sheets	1345	Bitumen	808	Soil	1242
6	Bare Soil	5029	Tiles	1260	Water	325
7	Bitumen	1330	Shadows	476	Residential	1268
8	Self-Blocking Bricks	3682	Meadows	824	Commercial	1244
9	Shadows	947	Bare Soil	820	Road	1252
10					Highway	1227
11					Railway	1235
12					Parking Lot 1	1233
13					Parking Lot 2	469
14					Tennis Court	428
15					Running Track	660
	Total	42776	Total	7456	Total	15029

decay to 0.001 according to three investigation experiments where weight decay is set to 1e-3, 1e-4, and 1e-5, respectively. We use the stochastic gradient descent (SGD) optimizer to update supernet parameters (θ). Following the search setting for CIFAR-10 in Darts [12], we set our SGD parameters with minor adjustment. The momentum and weight decay are set to 0.9 and 0.0003, respectively. The learning rate decays from 0.025 to 0.001 according to cosine annealing strategy [55]. We train the supernet at most 100 epochs. The first 20 epochs are warm-up stage, where we only optimize θ. From epoch 21, we update architecture parameters and supernet parameters alternately in each iteration.

4) Final Network Optimizing: For all three final networks found for three datasets, the same optimizing setting is used. Similar to the setup in the searching stage, we still crop $32 \times 32 \times$ L-sized patches as training and validation samples. In the final network, we only need to update network parameters, as shown in formula 9. The optimizer used here is the SGD optimizer. The batch size is set to 12. Inspired by the classical segmentation methods, deeplab series [56], [57],

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:42:04 UTC from IEEE Xplore. Restrictions apply.

TABLE II

COMPARISON EXPERIMENTAL RESULTS ON PAVIA UNIVERSITY USING 20 TRAINING SAMPLES EACH CATEGORY

Models	3D-LWNet	1-D Auto-CNN	3-D Auto-CNN	3D-ANAS	3D-ANAS +MS	3D-ANAS +OV	3D-ANAS +MS+OV
1	82.43	69.69	88.24	93.21	*93.35*	**93.39**	93.15
2	84.76	76.37	90.72	99.1	99.04	*99.21*	**99.43**
3	76.88	73.43	92.11	99.28	99.32	*99.52*	**99.71**
4	91.45	90.2	81.27	92.22	**93.05**	*92.65*	*92.65*
5	96.23	*96.54*	93.12	**100**	**100**	**100**	**100**
6	92.50	75.48	**98.47**	97.36	97.56	98.16	*98.18*
7	93.56	88.83	96.14	99.62	*99.69*	**100**	**100**
8	*96.01*	77.59	**96.84**	87.35	88.12	87.68	87.92
9	89.16	96.66	79.62	98.69	**99.35**	*99.13*	*99.13*
OA	87.10	77.65	91.16	96.53	96.69	*96.79*	**96.88**
AA	89.22	82.75	90.72	96.31	96.61	*96.64*	**96.69**
K	83.35	71.51	88.51	95.39	95.61	*95.73*	**95.85**

TABLE III

COMPARISON EXPERIMENTAL RESULTS ON PAVIA UNIVERSITY USING 30 TRAINING SAMPLES EACH CATEGORY

Models	3D-LWNet	1-D Auto-CNN	3-D Auto-CNN	3D-ANAS	3D-ANAS +MS	3D-ANAS +OV	3D-ANAS +MS+OV
1	82.32	76.43	89.70	93.21	93.3	*93.32*	**93.38**
2	88.84	81.9	97.92	**98.5**	98.22	*98.46*	98.32
3	83.74	74.04	92.31	*99.61*	*99.61*	**99.81**	**99.81**
4	**94.11**	*93.38*	71.22	98.09	98.02	*98.38*	98.35
5	*96.90*	96.13	95.12	**100**	**100**	**100**	**100**
6	92.10	81.81	96.96	99.8	*99.82*	**99.92**	**99.92**
7	95.46	88.23	95.99	**99.77**	*99.54*	*99.54*	**99.77**
8	93.19	74.05	94.98	96.55	96.5	*97.65*	**98.03**
9	92.43	*95.65*	80.64	**100**	**100**	**100**	**100**
OA	89.25	81.75	93.36	97.81	97.69	**97.94**	*97.92*
AA	91.01	84.62	90.54	98.39	98.34	*98.56*	**98.62**
K	86.05	76.55	91.50	97.1	96.94	**97.27**	*97.25*

TABLE IV

COMPARISON EXPERIMENTAL RESULTS ON PAVIA CENTER USING 20 TRAINING SAMPLES EACH CATEGORY

Models	3D-LWNet	1-D Auto-CNN	3-D Auto-CNN	3D-ANAS	3D-ANAS +MS	3D-ANAS +OV	3D-ANAS +MS+OV
1	99.61	*99.81*	99.56	**100**	**100**	**100**	**100**
2	91.85	80.9	87.79	**94.66**	*94.64*	**94.66**	94.61
3	86.77	89.04	82.98	*96.21*	95.59	**96.34**	96.11
4	92.85	73.53	98.85	99.89	*99.96*	**100**	**100**
5	95.53	90.05	95.83	97.6	97.77	*98.79*	**98.96**
6	80.97	95.53	93.22	99.8	99.86	*99.91*	**99.97**
7	85.66	84.23	94.94	97.88	97.89	*97.92*	**97.99**
8	85.06	98.17	95.12	*99.25*	*99.25*	**99.26**	99.23
9	91.34	98.89	85.29	*99.96*	**100**	**100**	**100**
OA	92.42	96.18	96.25	99.21	99.2	**99.28**	*99.27*
AA	89.96	90.02	92.62	*98.36*	98.33	**98.54**	**98.54**
K	89.41	94.6	94.71	98.88	98.87	**98.98**	*98.97*

we employ poly learning rate policy with power of 0.9 ($\text{lr} = \text{init_lr} \cdot (1 - (\text{iter}/(\text{max_iter}))^{\text{power}})$) to adjust the learning rate. The initial learning rate is set to 0.1, which is selected from [0.1, 0.01, 0.025] via investigation experiments. We optimize the final network for at most 600k iterations and evaluate the network on the validation set for every 100 iterations. This training process stops once the performance and loss of the validation set reach saturation. We use random crop, horizontal, and vertical flipping as well as random rotations $\in \{0°, 90°, 180°, 270°\}$ as data augmentation.

5) Inference Setting: For our pixel-to-pixel classification framework, we employ two inference augmentation strategies to further improve the performance. 1) Multiscale inference (termed as MS). As our 3-D-ANAS can accept patches of arbitrary spatial size as input, we crop the patches in three different scales, $16 \times 16 \times L$, $32 \times 32 \times L$, and $48 \times 48 \times L$, respectively. The final prediction result is obtained by averaging prediction results of three scales. 2) Overlap inference (termed as OV). During inference, we crop patches in a sliding window strategy then feed the cropped patches to trained networks. In the sliding window strategy, there are two hyper parameters, which are stride and window size. When stride is set to half of the window size, the adjacent patches have an overlap region, which is processed twice. In OV, we set stride to half of the window size and take the average result as the final prediction result. Based on the two inference augmentation strategies, we implement the inference process with four different strategies: 1) 3-D-ANAS, which uses a single scale 32 for inference and patches without overlapping regions; 2) 3-D-ANAS + MS, which uses the MS strategy for inference; 3) 3-D-ANAS + OV, inference using OV strategy; and 4) 3-D-ANAS + MS + OV, inference using both MS and OV strategies.

6) Experiment Design: To demonstrate the effectiveness of the proposed 3-D-ANAS, we distribute the labeled samples in two different proportions: 1) Distribution 1, we randomly select 20 labeled pixels from each category to build a training label map and randomly select 10 labeled pixels from each category to build the validation label map. Then the rest are reserved as test label maps. Taking Pavia University as an example, the training label map contains 180 labeled pixels, the validation label map consists of 90 labeled pixels, and the test label map has 42476 labeled pixels. 2) Distribution 2, we randomly choose 30 labeled pixels and 10 labeled pixels from each category to build the training label map and validation label map, respectively. As samples are randomly distributed, we run each experiment ten times and compare the average result to ensure the comparisons are fair and stable.

B. Comparison Experiments

In this section, we focus on comparing our proposed 3-D-ANAS with other CNN-based HSI classification methods. Here, three comparison methods published recently are employed. We conduct 3-D lightweight CNN (3-D-LWNet) based on the published code.[1] The results of 1-D Auto-CNN and 3-D Auto-CNN are obtained by using the published source code.[2] Tables II–VII list the comparison results and Figs. 8–10 show the corresponding visualization results.

The results of comparison experiments on Paiva University are listed in Tables II and III. The corresponding visual comparison results are shown in Fig. 8. From the comparison

[1]https://github.com/hkzhang91/LWNet

[2]https://github.com/YushiChen/Auto-CNN-HSI-Classification

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:42:04 UTC from IEEE Xplore. Restrictions apply.

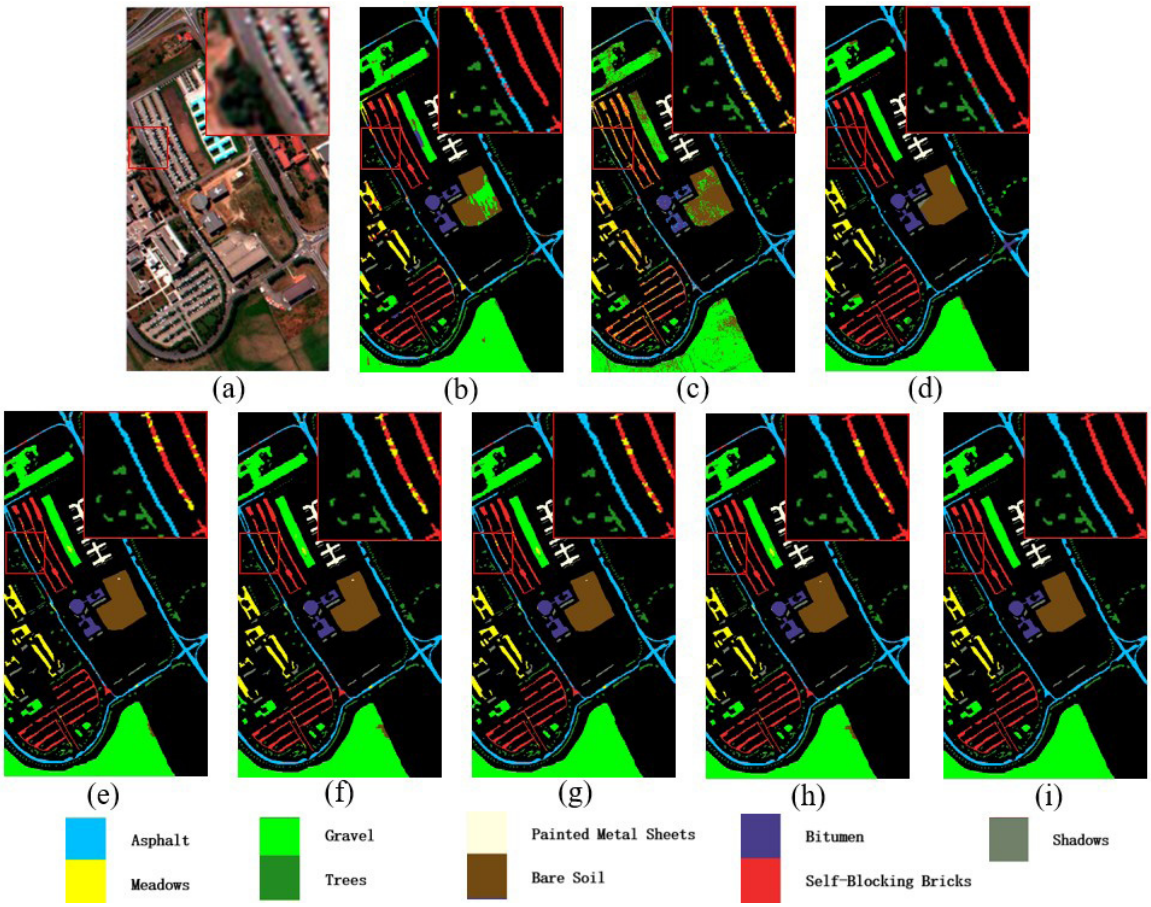


Fig. 8. Comparison experimental results on Pavia University using 30 training samples each category. (a) False color composite. (b) 3-D-LWNet OA = 89.25%. (c) 1-D Auto-CNN, OA = 81.75%. (d) 3-D Auto-CNN, OA = 93.36%. (e) 3-D-ANAS, OA = 97.81%. (f) 3-D-ANAS + MS, OA = 97.69%. (g) 3-D-ANAS + OV, OA = 97.94%. (h) 3-D-ANAS + MS + OV, OA = 97.92%. (i) Ground truth map.

TABLE V

COMPARISON EXPERIMENTAL RESULTS ON PAVIA CENTER USING 30 TRAINING SAMPLES EACH CATEGORY

Models	3D-LWNet	1-D Auto-CNN	3-D Auto-CNN	3D-ANAS	3D-ANAS +MS	3D-ANAS +OV	3D-ANAS +MS+OV
1	99.52	99.76	99.78	99.94	99.95	*99.96*	**99.97**
2	93.92	87.9	92.48	95.49	*95.56*	**95.59**	95.51
3	89.26	91.16	85.81	96.73	**97.09**	*96.86*	96.57
4	91.1	79.72	97.32	99.62	*99.77*	**99.81**	**99.81**
5	96.09	92.1	97.02	**99.69**	*99.59*	99.5	99.51
6	90.73	96.47	95.91	99.35	99.40	*99.65*	**99.73**
7	93.24	85.43	95.29	97.55	*97.57*	**97.60**	**97.60**
8	87.32	97.88	95.13	99.9	*99.92*	**99.93**	**99.93**
9	93.7	*98.58*	91.84	**100.0**	**100.0**	**100.0**	**100.0**
OA	94.22	96.79	96.99	99.47	99.49	**99.51**	*99.50*
AA	92.76	92.11	94.51	98.70	*98.76*	**98.77**	98.74
K	91.92	95.47	95.76	99.24	*99.28*	**99.30**	**99.30**

results, we can draw three conclusions: 1) compared with 1-D CNN-based methods, 3-D CNN-based methods generally have better performance. Among these methods compared in Tables II and III, the classification accuracy of 1-D Auto-CNN is lower than others. This is consistent with the statements in recent spectral-spatial HSI classification works. Using both spectral and spatial information is beneficial for improving classification accuracy. 2) our proposed 3-D-ANAS achieves much better performance compared with other CNN model-based methods and NAS-based approaches. For instance, by using 20 training samples per category, 3-D-ANAS achieves 96.53% overal accuracy (OA), 96.31% average accuracy (AA), and 95.39% Kappa coefficient (K), which are 5.37%, 5.59%, and 6.88% points higher than 3-D Auto-CNN, respectively. 3) Our inference augmentation strategy can further improve the performance.

As shown in Table II, using MS improves OA, AA, and K by 0.16%, 0.30%, and 0.22% points, respectively. Using OV improves OA, AA, and K by 0.26%, 0.33%, and 0.34% points, respectively. Compared with 3-D-ANAS which does not employ any inference augmentation strategies, 3-D-ANAS + MS + OV which uses both MS and OV strategies improves OA, AA, and K by 0.35%, 0.38%, and 0.46% points, respectively. In addition, it also demonstrates that the OV inference augmentation strategy is more useful than the MS inference augmentation strategy in improving classification accuracy. From Tables II and III, with the number of training samples per class increasing from 20 to 30, all methods obtain much higher classification accuracy. For instance, the OA of 3-D-LWNet increases from 87.10% to 89.25%. (With regarding transferring model from Salinas, 3-D-LWNet achieves 93.46% OA, 93.63% AA, and 91.43% K, which are highly competitive with 3-D Auto-CNN. Nevertheless,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:42:04 UTC from IEEE Xplore. Restrictions apply.

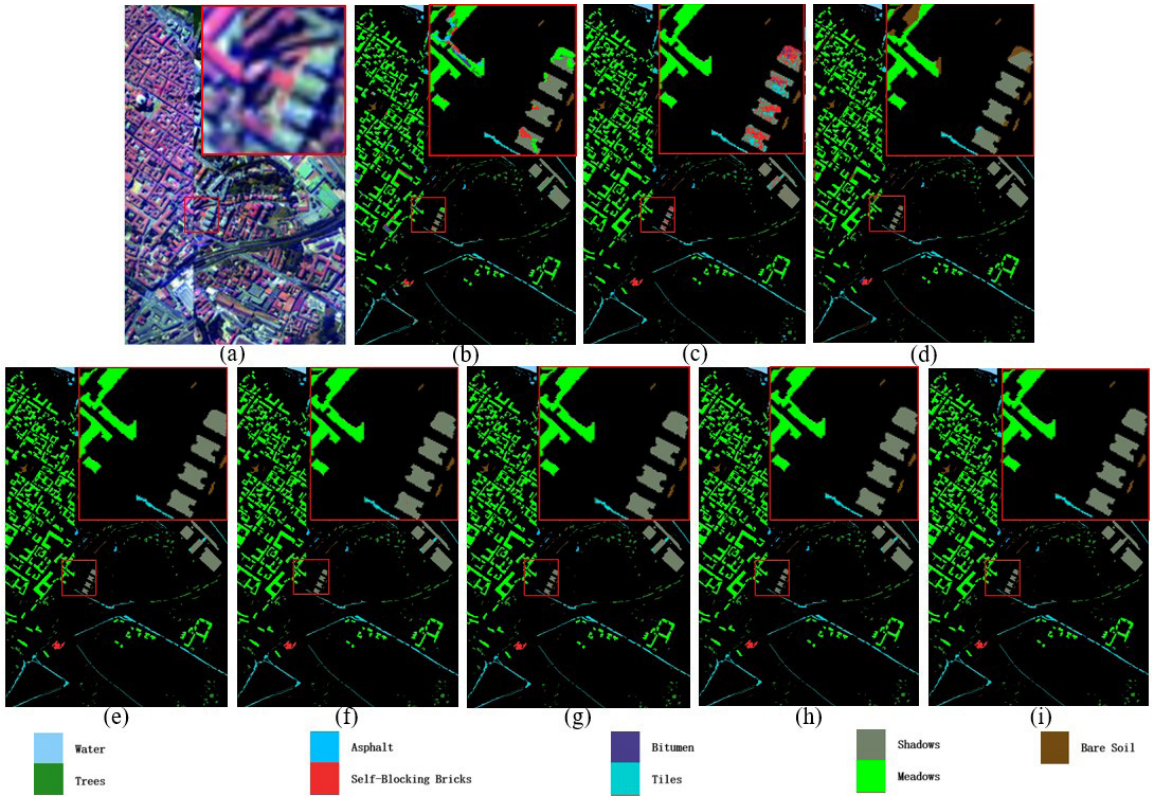


Fig. 9. Comparison experimental results on Pavia Center using 30 training samples each category. (a) False color composite. (b) 3-D-LWNet OA = 94.22%. (c) 1-D Auto-CNN, OA = 96.79%. (d) 3-D Auto-CNN, OA = 96.99%. (e) 3-D-ANAS, OA = 99.47%. (f) 3-D-ANAS + MS, OA = 99.49%. (g) 3-D-ANAS + OV, OA = 99.51%. (h) 3-D-ANAS + MS + OV, OA = 99.50%. (i) Ground truth map.

TABLE VI

COMPARISON EXPERIMENTAL RESULTS ON HOUSTON UNIVERSITY USING 20 TRAINING SAMPLES EACH CATEGORY

Models	3D-LWNet	1-D Auto-CNN	3-D Auto-CNN	3D-ANAS	3D-ANAS +MS	3D-ANAS +OV	3D-ANAS +MS+OV
1	79.16	69.98	**84.01**	82.06	82.47	82.64	*83.21*
2	71.61	60.19	85.65	79.17	*79.33*	*79.33*	**79.58**
3	94.4	77.30	93.86	96.10	96.40	**97.45**	*96.85*
4	74.25	49.02	66.77	81.71	*82.62*	**83.36**	**83.36**
5	90.37	83.09	93.83	94.39	94.47	*94.72*	**95.05**
6	84.92	50.46	80.43	**99.32**	*98.98*	97.97	*98.98*
7	**84.73**	30.93	72.21	81.58	82.23	*83.44*	82.71
8	52.01	50.53	70.96	70.29	71.66	*71.91*	**72.16**
9	70.47	46.58	71.18	76.27	76.27	*76.6*	**77.17**
10	92.15	69.73	96.43	96.49	96.74	*97.08*	**97.83**
11	*89.39*	50.06	**93.73**	73.36	72.95	72.61	73.78
12	60.04	63.44	**87.95**	*75.39*	75.06	74.40	74.40
13	86.99	53.35	84.90	90.66	*93.17*	**95.9**	**95.9**
14	*92.99*	76.73	*92.99*	**100.0**	**100.0**	**100.0**	**100.0**
15	**92.83**	54.00	88.30	86.51	87.3	90.16	*90.95*
OA	78.95	58.35	83.37	83.15	83.53	*83.96*	**84.24**
AA	81.09	59.03	84.21	85.55	85.98	*86.50*	**86.79**
K	77.32	55.18	82.07	81.81	82.22	*82.68*	**82.99**

TABLE VII

COMPARISON EXPERIMENTAL RESULTS ON HOUSTON UNIVERSITY USING 30 TRAINING SAMPLES EACH CATEGORY

Models	3D-LWNet	1-D Auto-CNN	3-D Auto-CNN	3D-ANAS	3D-ANAS +MS	3D-ANAS +OV	3D-ANAS +MS+OV
1	84.81	72.25	**87.5**	86.16	86.57	86.57	*87.22*
2	80.22	63.96	77.91	84.31	84.72	*87.5*	**87.75**
3	**93.45**	76.61	*92.74*	86.06	85.46	85.46	84.26
4	79.74	51.29	72.65	79.74	79.57	*81.05*	**81.30**
5	90.9	82.25	96.14	97.11	**97.44**	*97.28*	97.19
6	81.44	55.32	84.86	87.46	*88.81*	**89.83**	*88.81*
7	**87.83**	34.2	73.03	77.79	77.79	*81.10*	80.45
8	63.69	62.11	76.64	78.75	78.83	*79.16*	**79.32**
9	**77.5**	49.84	71.1	73.57	74.55	74.80	*75.45*
10	93.26	64.48	**96.67**	95.15	95.32	*96.41*	95.99
11	91.04	50.23	92.31	91.45	92.70	*94.36*	**95.77**
12	79.4	62.47	**91.48**	87.61	88.11	89.36	*89.86*
13	90.41	50.92	85.8	*91.12*	*91.12*	**94.08**	**94.08**
14	90.65	76.07	90.65	**99.75**	**99.75**	**99.75**	*99.25*
15	92.47	62.94	88.85	93.65	94.28	*95.40*	**95.71**
OA	84.17	60.36	84.45	86.16	86.51	*87.67*	**87.82**
AA	85.12	61.00	85.22	87.31	87.67	*88.81*	**88.83**
K	82.96	57.35	83.25	85.03	85.42	*86.67*	**86.83**

as other methods do not use any transfer learning strategies, here we show the results of 3-D-LWNet without using cross-sensor and cross-modal transfer learning strategies for a fair comparison.) Our proposed 3-D-ANAS still has a better performance compared with others. Unexpectedly, in Table III, using MS during inference slightly degrades the final performance. We conjecture this because we only use a single scale during training. To save space, instead of showing both corresponding results of Tables II and III, we illustrate the visual results of using 30 training samples in each category in Fig. 8. To clearly show the difference, we place a local enlarged patch in the top right corner of each

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:42:04 UTC from IEEE Xplore. Restrictions apply.

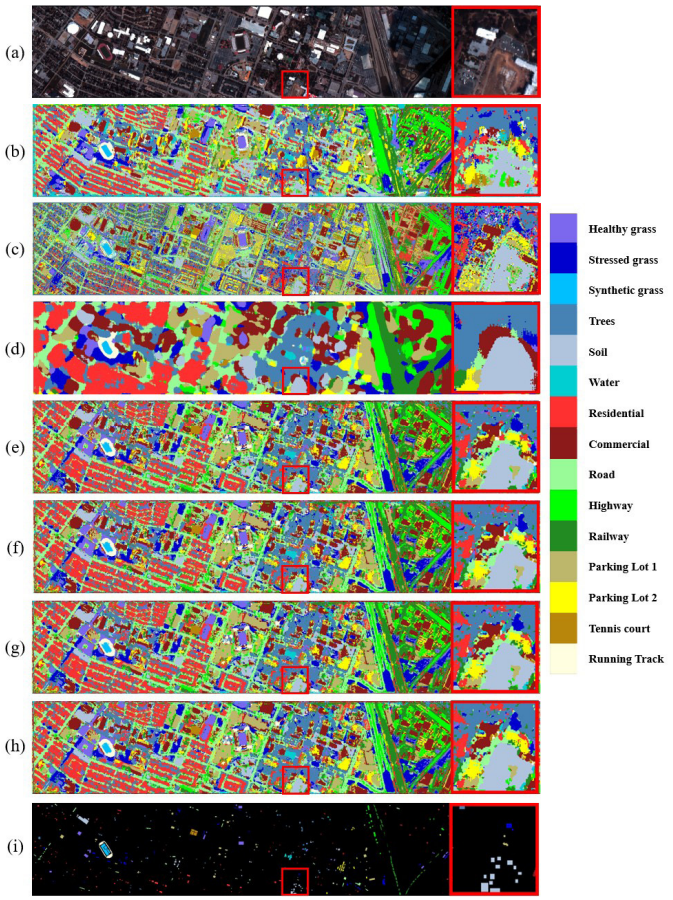


Fig. 10. Comparison experimental results on Houston University using 30 training sample each category. (a) False color composite. (b) 3-D-LWNet OA = 84.17%. (c) 1-D Auto-CNN, OA = 60.36%. (d) 3-D Auto-CNN, OA = 84.45%. (e) 3-D-ANAS, OA = 86.16%. (f) 3-D-ANAS + MS, OA = 86.51%. (g) 3-D-ANAS + OV, OA = 87.67%. (h) 3-D-ANAS + MS + OV, OA = 87.82%. (i) Ground truth map.

result map. From the local enlarged patches, we can see that fewer pixels were misclassified in the results of 3-D-ANAS, 3-D-ANAS + MS, 3-D-ANAS + OV, and 3-D-ANAS + MS + OV than in the other methods. In the local enlarged region, part of Asphalt pixels (class 1, cyan) is misclassified as Self-Blocking Bricks (class 8, red) by 3-D-LWNet and 3-D Auto-CNN, most of the pixels that belong to Self-Blocking Bricks are misclassified as Meadows (class 2, green) by 1-D Auto-CNN. In contrast to this, in the results of 3-D-ANAS, all pixels of Asphalt are correctly classified and a small number of pixels belonging to Self-Blocking Bricks are misclassified.

C. Effectiveness of 3-D-ANAS

Tables II and III collect the comparison results on Pavia Center and Fig. 8 exhibits the visual results. Compared with Pavia University, all seven methods achieve higher accuracy. The network found by 3-D-ANAS still shows the best performance. At the Pavia Center, the improvement brought by using inference augmentation is lower than in Pavia University. We conjecture this is due to the relatively high accuracy already achieved by 3-D-ANAS and the limited space for further improvement. Even so, using OV inference augmentation strategy still improves OA, AA and K by 0.07%, 0.08%, and 0.1% points in using 20 training samples each category (Table IV), 0.04%, 0.07%, and 0.06% points in using 30 training samples each category (Table V). In Fig. 9, it can be clearly seen that 3-D-ANAS has almost no misclassified pixels in the local enlarged regions.

TABLE VIII
EFFECTIVENESS OF 3-D-ANAS

Datasets	Acc.	3D-ANAS	3D-ANAS R_op	3D-ANAS R_topo	3D-ANAS R_W↑	3D-ANAS R_W↓
Pavia U	OA	97.81	97.42	96.84	97.43	97.18
	AA	98.39	98.01	97.62	98.04	97.90
	K	97.10	96.30	95.84	96.82	96.21
Pavia C	OA	99.47	99.13	98.91	99.03	-
	AA	98.70	98.22	97.79	97.53	-
	K	99.24	98.79	98.68	98.64	-
Houston U	OA	86.16	85.78	85.32	85.89	85.79
	AA	87.31	86.96	86.57	87.01	86.93
	K	85.03	84.63	84.31	84.77	84.65

TABLE IX
COMPARISON EXPERIMENTS BETWEEN DIFFERENT SEARCH SPACES

Datasets	Search spaces	de.	asym.	OA	AA	K
Pavia U.	3D-sym-ud	✗	✗	96.91	97.24	95.91
	3D-sym-d	✓	✗	97.17	97.71	96.26
	3D-asym-ud	✗	✓	97.45	98.11	97.03
	3D-asym-d	✓	✓	97.81	98.39	97.10
Pavia C.	3D-sym-ud	✗	✗	99.02	98.19	98.61
	3D-sym-d	✓	✗	99.13	98.09	98.76
	3D-asym-ud	✗	✓	99.31	98.55	99.03
	3D-asym-d	✓	✓	99.47	98.70	99.24
Houston U.	3D-sym-ud	✗	✗	85.21	86.32	84.19
	3D-sym-d	✓	✗	85.63	86.91	84.59
	3D-asym-ud	✗	✓	85.91	87.21	84.76
	3D-asym-d	✓	✓	86.16	87.31	85.03

Tables VI and VII and Fig. 10 illustrates the comparison results on Houston University. Compared with the last two datasets, Houston University contains more spectral bands and covers more classes. Correspondingly, all methods achieve lower classification accuracy on this dataset. The classification performance of the different methods varied widely. For example, in the experiments using 20 training samples per class, the lowest OA, AA, and K are 58.32%, 59.03%, and 55.18%, while the highest OA, AA, and K are 84.24%, 86.79%, and 82.99%. The gap between these two sets of results is about 30% points. 3-D Auto-CNN and 3-D-ANAS have much better performances compared with others. Without using any inference augmentation strategy, 3-D-ANAS achieves 83.15% OA and 81.81% K, which are slightly lower than 3-D Auto-CNN. Yet, AA of 3-D-ANAS is 85.55%, which is significantly higher than that of 3-D Auto-CNN. When at least one inference augmentation strategy is used, 3-D-ANAS slightly beats 3-D Auto-CNN in all three metrics. Overall, in the experiment setting that 20 training samples are used

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:42:04 UTC from IEEE Xplore. Restrictions apply.

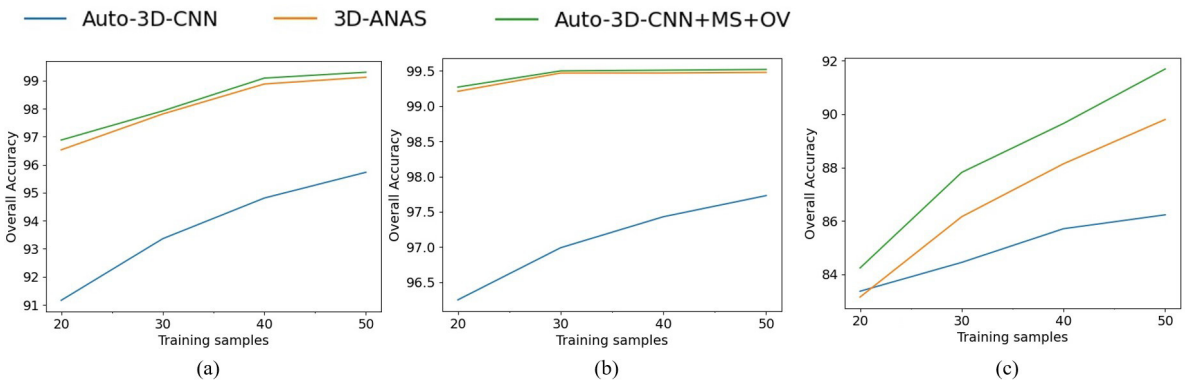


Fig. 11. Classification accuracy curves with a number of training sizes. (a) Pavia university. (b) Pavia center. (c) Houston university.

for each category, 3-D-ANAS achieves comparable results compared with 3-D Auto-CNN. When the training samples are increased to 30 for each class, 3-D-ANAS reveals obvious advantages. From Fig. 10, we can see that the classification map of 1-D Auto-CNN clearly shows the outline of the structure of different buildings. For instance, the dark red part (Commercial, Class 8) in the local enlarged region. However, at the same time, a lot of pixels are misclassified by 1-D Auto-CNN. These misclassified pixels are distributed throughout the resulting map and look like impulse noise, resulting in a relatively poor visual effect. On the contrary, 3-D Auto-CNN shows a very smooth result, where the outline of the structure is lost. 3-D-ANAS achieves a relatively better balance between showing good visual effects and keeping outline structures. In the Houston University dataset, performance improvement is not significant compared with 3-D CNN. We conjecture this is caused by the fact that classification in the Houston University dataset is difficult. Compared with the Pavia University and the Pavia Center datasets, the Houston University dataset contains more spectral bands and covers more categories. As a result, classification in the Houston University dataset is tougher. The network designed by our 3-D-ANAS has a higher capacity than 3-D CNN, but its capacity has not been brought into full play in the Houston University dataset when it is trained with few training samples (20 samples each category). When we increase training samples to 30 in each category, our proposed method shows significant advantages. For a more comprehensive assessment, we show the classification accuracy curves in Fig. 11.

D. Architecture Analysis

In this section, we focus on analyzing the architectures found by 3-D-ANAS. As the three datasets have a different spectral resolution, spatial resolution and cover different objects, we separately perform the architecture search for each dataset. The three found architectures are shown in Fig. 12. From Fig. 12, we note that the three architectures are different in both topological structure and selecting operations. Nevertheless, these three architectures also share some common characteristics:

1) The asymmetric convolution operation plays a major role in the final selection operations. As we discussed in Section III-A, the search space we build for searching for inner topological structures contains both symmetric convolution operations which have the same spatial and spectral receptive field and asymmetric convolution operations which have different spatial and spectral receptive fields. Even so, in most cases, 3-D-ANAS still tends to build a network that takes asymmetric convolution operations as main operations and uses symmetric convolution operations as complementary operations. The proportion of asymmetric convolution operations in the final network designed for Paiva University is 58.33%, for the Pavia Center is 66.67% and for the Houston University is 41.61%. The proportion of symmetric convolution operations are 16.67%, 16.67%, and 50%. We believe this result verifies our previous analysis. The sizes of kernels along the spatial dimension and spectral dimension should be different as HSIs have relatively low spatial resolution and extremely high spectral resolution.
2) Asymmetric convolution operations which have big spatial receptive field mainly distribute in shallow layers and asymmetric convolution operations with large spectral receptive field mainly locate in deep layers. Especially in the network designed for Pavia University, there are seven asymmetric convolution operations. All three asymmetric convolution operations with large spatial kernel sizes are in the first cell and 3/4 asymmetric convolution operations with a large spectral kernel size are positioned in the last cell.
3) 3-D-ANAS prefers to set different widths for different layers. Among the three architectures, two of them have different widths for different layers.

As shown in Fig. 12, the networks found by our proposed 3-D-ANAS have many fragmented branches. According to study results in [52], these fragmentation structures are beneficial for improving classification accuracy. To verify whether 3-D-ANAS improves the accuracy by actually searching for efficient architectures or by simply integrating various branches and convolution operations, we manually modify the architecture automatically designed by our proposed 3-D-ANAS in three different ways. Specifically, the three modifications are: 1) randomly replacing two

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:42:04 UTC from IEEE Xplore. Restrictions apply.

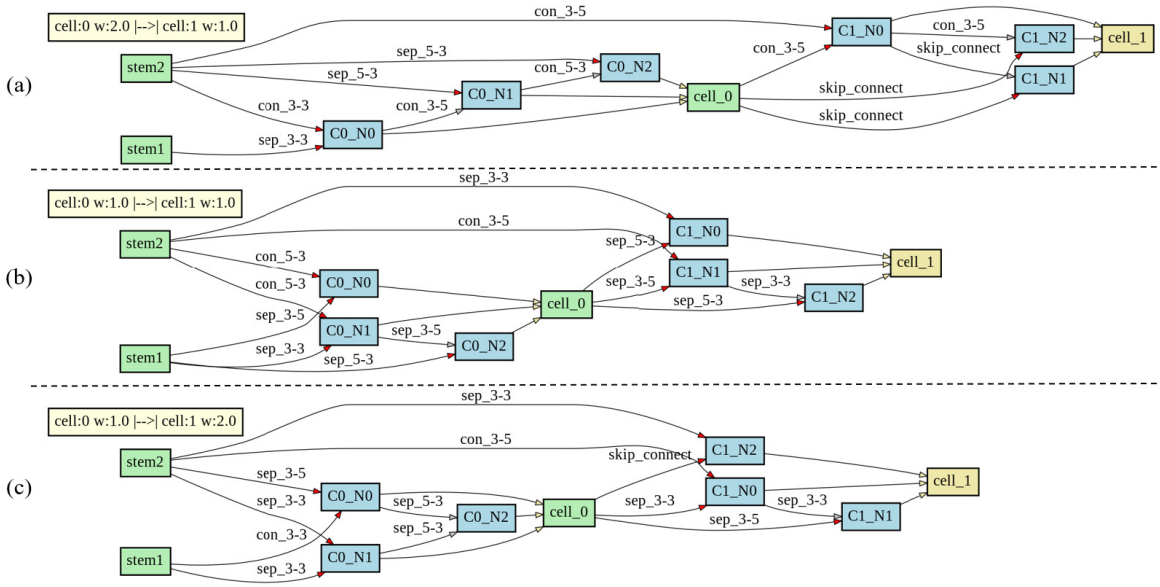


Fig. 12. Final architectures found for three datasets. (a) Pavia university. (b) Pavia center. (c) Houston university.

3-D-ANAS selected operation with others (termed as R_op). For instance, replacing sep_5-3 with Con_5-3; 2) changing the connection relationships between the nodes inside each cell (termed as R_topo); 3) increasing the widths (R_W↑) all widths of cells are set to $\gamma^1 W$. Decreasing widths (R_W↓), all widths of the cell are set to $\gamma^0 W$.

Following the three modification methods, we modify the found architectures several times. Each time different operations and connections in different nodes are modified. According to experimental results, we find that the modified architectures generally yield lower performance. The results are listed in Table VIII, from which we can draw the following conclusions:

1) 3-D-ANAS is able to select a proper operation for each path. Comparing the results in the fourth column with those in the third column, we can see that replacing the automatically select operations with others decreases accuracy on all three datasets.
2) 3-D-ANAS has the ability to design efficient topological structures. For instance, at the Pavia University, the modification in topological structure drops OA, AA, and K to 96.84%, 97.62%, and 95.84%, respectively.
3) Compared with randomly replacing selected operations, changing the network architecture impacts the efficiency of the network more seriously.
4) Searching widths for each cell is beneficial for improving classification accuracy. Using either R_W↑ or R_W↓ shows a slightly negative on the performance. With manually designed widths, the performance is still decent. In our opinion, this shows that searching widths can further improve the performance.

Overall, the experimental results demonstrate that 3-D-ANAS does find an efficient topological structure and selects the proper operation, rather than simply integrating a network with many branches and various convolution operations. The fact that a slight perturbation to the found architecture deteriorates the accuracy indicates that the found architecture is indeed a local optimum in the architecture search space.

E. Effectiveness of 3-D Asymmetric Search Space

By in-depth analysis of HSIs, we find that spectral resolution and spatial resolution are not consistent in HSIs. The spatial resolution is relatively low while the spectral resolution is extremely high. Based on this discovery, we conjecture that spatial dimension and spectral dimension should be processed differently. Therefore, we build a 3-D asymmetric decomposition search space, which contains three kinds of convolution operations: 1) asymmetric convolution operation where kernel size along spatial dimension is larger than that along spectral dimension; 2) asymmetric convolution operation which is the opposite of 1). 3) decomposed convolution operation where, the kernel size along spatial and spectral dimension are the same. In Section IV-D, the search results have demonstrated that 3-D-ANAS does prefer to select 3-D asymmetric convolution operations, which partly supports our conjecture. In this section, to further evaluate the effectiveness of 3-D asymmetric decomposition search space, we build three other different search spaces, including 1) 3-D symmetrically undecomposed search space (3-D-asym); 2) 3-D symmetrically decomposed search space (3-D-sym-d); and 3) 3-D asymmetrically undecomposed search space (3-D-asym-ud). The specific components of these three search spaces are listed as follows:

3-D Symmetrically Undecomposed Search Space:

1) *Con_3:* LReLU-Conv($3 \times 3 \times 3$)-BN;
2) *Con_5:* LReLU-Conv($5 \times 5 \times 5$)-BN;
3) *Con_7:* LReLU-Conv($7 \times 7 \times 7$)-BN;
4) *Sep_3:* LReLU-Sep($3 \times 3 \times 3$)-BN;
5) *Sep_5:* LReLU-Sep($5 \times 5 \times 5$)-BN;

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:42:04 UTC from IEEE Xplore. Restrictions apply.

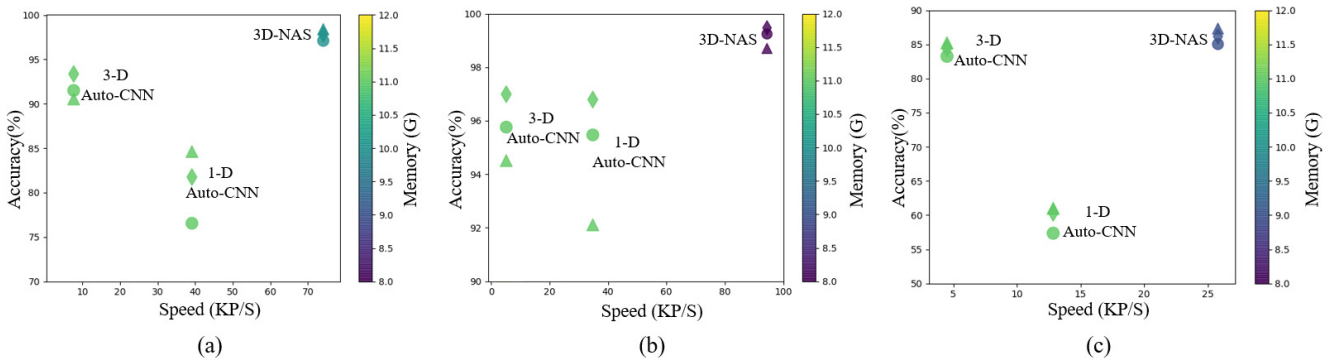


Fig. 13. Final architectures found for three datasets. (a) Pavia University. (b) Pavia Center. (c) Houston University. OA, AA, and K are marked with rhombic, triangular and circular icons.

6) *Sep_7:* LReLU-Sep($7 \times 7 \times 7$)-BN;
7) *Skip_Connection:* $f(x) = x$;
8) discarding: $f(x) = 0$.

3-D Symmetrically Decomposed Search Space:

1) *Dcon_3-3:* LReLU-Conv($1 \times 3 \times 3$)-Conv($3 \times 1 \times 1$)-BN;
2) *Dcon_5-5:* LReLU-Conv($1 \times 5 \times 5$)-Conv($5 \times 1 \times 1$)-BN;
3) *Dcon_7-7:* LReLU-Conv($1 \times 7 \times 7$)-Conv($7 \times 1 \times 1$)-BN;
4) *Dsep_3-3:* LReLU-Sep($1 \times 3 \times 3$)-Sep($3 \times 1 \times 1$)-BN;
5) *Dsep_5-5:* LReLU-Sep($1 \times 5 \times 5$)-Sep($5 \times 1 \times 1$)-BN;
6) *Dsep_7-7:* LReLU-Sep($1 \times 7 \times 7$)-Sep($7 \times 1 \times 1$)-BN;
7) *Skip_Connection:* $f(x) = x$;
8) discarding: $f(x) = 0$.

3-D Asymmetrically Undecomposed Search Space:

1) *Udcon_3-3:* LReLU-Conv($1 \times 3 \times 3$)-Conv($3 \times 1 \times 1$)-BN;
2) *Udcon_5-5:* LReLU-Conv($1 \times 5 \times 5$)-Conv($5 \times 1 \times 1$)-BN;
3) *Udcon_7-7:* LReLU-Conv($1 \times 7 \times 7$)-Conv($7 \times 1 \times 1$)-BN;
4) *Udsep_3-3:* LReLU-Sep($1 \times 3 \times 3$)-Sep($3 \times 1 \times 1$)-BN;
5) *Udsep_5-5:* LReLU-Sep($1 \times 5 \times 5$)-Sep($5 \times 1 \times 1$)-BN;
6) *Udsep_7-7:* LReLU-Sep($1 \times 7 \times 7$)-Sep($7 \times 1 \times 1$)-BN;
7) *Skip_Connection:* $f(x) = x$;
8) discarding: $f(x) = 0$.

In the 3-D convolution search space, the 3-D convolution operations are common 3-D convolutions, which contains two kinds of convolutions, conventional 3-D convolution and separable 3-D convolution, each of which contains three specific settings corresponding to different kernel sizes. In the 3-D symmetric search space, each conventional 3-D convolution is decomposed into a sequence of two convolutions corresponding to spatial convolution and spectral convolution.

The comparison results between these three search spaces are listed in Table IX. The architectures found by using the 3-D convolution search space have lower accuracy compared with the architectures designed by using the other two search space. Architectures found by using symmetric search space display worse performance compared with architectures obtained by adopting asymmetric search space. For Pavia University, decomposing 3-D convolution in 3-D convolution search space improves overall accuracy by about 0.3% points and introducing and asymmetric structure improves the overall accuracy by about 0.6% points. From 3-D-sym-ud to 3-D-asym-d, OA, AA, and K are increased by 0.9%, 1.15%, and 1.21% points, respectively. Among the four search space, we recommend to use 3-D-asym-d. It adopts both design skills, including decomposed 3-D convolution and asymmetric structure. Using decomposed 3-D convolution makes the search space has fewer parameters and deeper structure. The asymmetric structure is more aligned with the structure characteristics of HSI.

F. Inference Speed

Previous DL-based HSI classification methods usually employ a patch-to-pixel classification framework, which is relatively inefficient in computation overlapping regions of adjacent patches that need to be repeatedly computed. To overcome this issue, in our 3-D-ANAS, we employ a pixel-to-pixel classification framework instead. In this section, we compare the inference speed of these two classification frameworks and show the comparison results in Fig. 13.

In the comparison experiments of using 30 training samples for each class, our proposed 3-D-ANAS achieves the best performances on all three datasets, while having the fastest inference speed. This is especially on the Paiva Center [Fig. 13(b)], 3-D-ANAS has 94.37 KP/S (thousand pixels per second) inference speed, which is about 18.15× that of 3-D Auto-CNN and 2.71× that of 1-D Auto-CNN. It is worth noting that, 1-D Auto-CNN-based HSI classification is also a pixel-to-pixel classification, without duplicate processing pixels. However, the inference speed of 1-D Auto-CNN is still lower than our 3-D-ANAS. We conjecture this is caused by the fact that the operate mode in 3-D-ANAS is more aligned with Pytorch frame [58].

V. Conclusion

Most DL HSI classification methods are based on manually designed networks, in which a lot of human efforts are spent on designing state-of-the-art neural network architectures. To overcome this inhibitor, we have proposed 3-D-ANAS, an asymmetric neural architecture search algorithm for HSI classification. The proposed 3-D-ANAS has a hierarchical search space, including an inner search space and an outer

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:42:04 UTC from IEEE Xplore. Restrictions apply.

search space. In the inner search space, a 3-D asymmetric search space is built according to characteristics of HSIs for searching for architectures. In the outer search space, three candidate paths with different widths are provided for searching for width for each layer. In addition, to improve the inference speed and overcome the problems of repetitive operations and scale sensitivity in the patch-to-pixel classification framework, 3-D-ANAS employs a new classification framework, which is a pixel-to-pixel classification framework.

We conducted comparison experiments on three challenging public HSI datasets and compared our proposed 3-D-ANAS with other DL-based HSI classification methods, including both manually designed CNN-based methods (3-D-LWNet) and automatic design CNN-based methods (1-D Auto-CNN and 3-D Auto-CNN). The results of comparison experiments have demonstrated our proposed 3-D-ANAS achieve competitive performance with the other methods. In addition, we have organized abundant ablation experiments to verify the effectiveness of our proposed 3-D asymmetric search space, 3-D-ANAS, and pixel-to-pixel classification framework. Results of the ablation study show that our 3-D-ANAS does find a local optimum architecture in the architecture search space and pixel-to-pixel classification is beneficial for improving inference speed. Compared with the patch-to-pixel classification framework, the pixel-to-pixel classification framework improves inference speed up to $18\times$.

In our future work, we will focus on developing more efficient NAS algorithms to automatically design efficient architectures for HSI classification. In addition, we will pay more attention to study the relationship between the found architectures and characteristics of HSIs.

REFERENCES

[1] J. Zhu, J. Hu, S. Jia, X. Jia, and Q. Li, "Multiple 3-D feature fusion framework for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 4, pp. 1873–1886, Apr. 2018.

[2] A. J. Brown *et al.*, "Hydrothermal formation of clay-carbonate alteration assemblages in the Nili Fossae region of Mars," *Earth Planet. Sci. Lett.*, vol. 297, nos. 1–2, pp. 174–182, Aug. 2010.

[3] A. J. Brown, B. Sutter, and S. Dunagan, "The MARTE VNIR imaging spectrometer experiment: Design and analysis," *Astrobiology*, vol. 8, no. 5, pp. 1001–1011, Oct. 2008.

[4] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.

[5] Y. Chen, X. Zhao, and X. Jia, "Spectral–spatial classification of hyperspectral data based on deep belief network," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 8, no. 6, pp. 2381–2392, Jun. 2015.

[6] H. Zhang, Y. Li, Y. Zhang, and Q. Shen, "Spectral–spatial classification of hyperspectral imagery using a dual-channel convolutional neural network," *Remote Sens. Lett.*, vol. 8, no. 5, pp. 438–447, May 2017.

[7] Y. Li, H. Zhang, and Q. Shen, "Spectral–spatial classification of hyperspectral imagery with 3D convolutional neural network," *Remote Sens.*, vol. 9, no. 1, p. 67, Jan. 2017.

[8] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.

[9] L. Shu, K. McIsaac, and G. R. Osinski, "Hyperspectral image classification with stacking spectral patches and convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 10, pp. 5975–5984, Oct. 2018.

[10] H. Zhang, Y. Li, Y. Jiang, P. Wang, Q. Shen, and C. Shen, "Hyperspectral classification based on lightweight 3-D-CNN with transfer learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 8, pp. 5813–5828, Aug. 2019.

[11] C. Liu *et al.*, "Auto-DeepLab: Hierarchical neural architecture search for semantic image segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 82–92.

[12] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable architecture search," 2019, *arXiv:1806.09055*. [Online]. Available: https://arxiv.org/abs/1806.09055

[13] H. Zhang, Y. Li, H. Chen, and C. Shen, "Memory-efficient hierarchical neural architecture search for image denoising," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 3657–3666.

[14] P. Liu, M. D. El Basha, Y. Li, Y. Xiao, P. C. Sanelli, and R. Fang, "Deep evolutionary networks with expedited genetic algorithms for medical image denoising," *Med. Image Anal.*, vol. 54, pp. 306–315, May 2019.

[15] X. Chu, B. Zhang, H. Ma, R. Xu, and Q. Li, "Fast, accurate and lightweight super-resolution with neural architecture search," 2019, *arXiv:1901.07261*. [Online]. Available: http://arxiv.org/abs/1901.07261

[16] Y. Chen, K. Zhu, L. Zhu, X. He, P. Ghamisi, and J. A. Benediktsson, "Automatic design of convolutional neural network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 7048–7066, Sep. 2019.

[17] Z. Lin, Y. Chen, X. Zhao, and G. Wang, "Specrral-spatial classification of hyperspectral image using autoencoders," in *Proc. 9th Int. Conf. Inf. Commun. Signal Process.*, Dec. 2013, pp. 1–5.

[18] G. Licciardi, P. R. Marpu, J. Chanussot, and J. A. Benediktsson, "Linear versus nonlinear pca for the classification of hyperspectral data based on the extended morphological profiles," *IEEE Geosci. Remote Sens. Lett.*, vol. 9, no. 3, pp. 447–457, May 2011.

[19] X. Ma, H. Wang, and J. Geng, "Spectral–spatial classification of hyperspectral image based on deep auto-encoder," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 9, no. 9, pp. 4073–4085, Sep. 2016.

[20] H. Zhang and Y. Li, "Spectral–spatial classification of hyperspectral imagery based on deep convolutional network," in *Proc. Int. Conf. Orange Technol. (ICOT)*, Dec. 2016, pp. 44–47.

[21] S. Mei, J. Ji, Q. Bi, J. Hou, Q. Du, and W. Li, "Integrating spectral and spatial information into deep convolutional neural networks for hyperspectral classification," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2016, pp. 5067–5070.

[22] K. Makantasis, K. Karantzalos, A. Doulamis, and N. Doulamis, "Deep supervised learning for hyperspectral data classification through convolutional neural networks," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2015, pp. 4959–4962.

[23] J. Yue, W. Zhao, S. Mao, and H. Liu, "Spectral–spatial classification of hyperspectral images using deep convolutional neural networks," *Remote Sens. Lett.*, vol. 6, no. 6, pp. 468–477, Jun. 2015.

[24] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Oct. 2016.

[25] H. Wu and S. Prasad, "Convolutional recurrent neural networks for hyperspectral data classification," *Remote Sens.*, vol. 9, no. 3, p. 298, Mar. 2017.

[26] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 8697–8710.

[27] N. Wang *et al.*, "NAS-FCOS: Fast neural architecture search for object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 11943–11951.

[28] M. Suganuma, M. Ozay, and T. Okatani, "Exploiting the potential of standard convolutional autoencoders for image restoration by evolutionary search," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4771–4780.

[29] D. Song, C. Xu, X. Jia, Y. Chen, C. Xu, and Y. Wang, "Efficient residual dense block search for image super-resolution," in *Proc. AAAI*, 2020, pp. 12007–12014.

[30] E. D. Cubuk, B. Zoph, D. Mané, V. Vasudevan, and Q. V. Le, "AutoAugment: Learning augmentation strategies from data," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 113–123.

[31] P. Ramachandran, B. Zoph, and Q. V. Le, "Searching for activation functions," 2017, *arXiv:1710.05941*. [Online]. Available: https://arxiv.org/abs/1710.05941

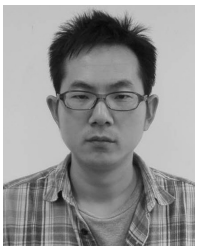
[32] Z. Zhong, J. Yan, W. Wu, J. Shao, and C.-L. Liu, "Practical block-wise neural network architecture generation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 2423–2432.

[33] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized evolution for image classifier architecture search," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 4780–4789.

[34] H. Liu, K. Simonyan, O. Vinyals, C. Fernando, and K. Kavukcuoglu, "Hierarchical representations for efficient architecture search," 2018, *arXiv:1711.00436*. [Online]. Available: https://arxiv.org/abs/1711.00436

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:42:04 UTC from IEEE Xplore. Restrictions apply.

[35] C. Zhang, M. Ren, and R. Urtasun, "Graph HyperNetworks for neural architecture search," 2018, *arXiv:1810.05749*. [Online]. Available: http://arxiv.org/abs/1810.05749
[36] T. Elsken, J. H. Metzen, and F. Hutter, "Efficient multi-objective neural architecture search via Lamarckian evolution," 2018, *arXiv:1804.09081*. [Online]. Available: http://arxiv.org/abs/1804.09081
[37] H. Pham, M. Y. Guan, B. Zoph, Q. V. Le, and J. Dean, "Efficient neural architecture search via parameter sharing," 2018, *arXiv:1802.03268*. [Online]. Available: https://arxiv.org/abs/1802.03268
[38] H. Cai, T. Wang, Z. Wu, K. Wang, J. Lin, and S. Han, "On-device image classification with proxyless neural architecture search and quantization-aware fine-tuning," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshop (ICCVW)*, Oct. 2019, pp. 1–5.
[39] D. Golovin, B. Solnik, S. Moitra, G. Kochanski, J. Karro, and D. Sculley, "Google vizier: A service for black-box optimization," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2017, pp. 1487–1495.
[40] L.-C. Chen *et al.*, "Searching for efficient multi-scale architectures for dense image prediction," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 8713–8724.
[41] K. Kandasamy, W. Neiswanger, J. Schneider, B. Poczos, and E. Xing, "Neural architecture search with Bayesian optimisation and optimal transport," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 2020–2029.
[42] J. Wang, J. Xu, and X. Wang, "Combination of hyperband and Bayesian optimization for hyperparameter optimization in deep learning," 2018, *arXiv:1801.01596*. [Online]. Available: http://arxiv.org/abs/1801.01596
[43] X. Ning, Y. Zheng, T. Zhao, Y. Wang, and H. Yang, "A generic graph-based neural architecture encoding scheme for predictor-based NAS," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 189–204.
[44] C. Zhang, M. Ren, and R. Urtasun, "Graph hypernetworks for neural architecture search," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–17.
[45] A. L. Maas, A. Y. Hannun, and A. Y. Ng, "Rectifier nonlinearities improve neural network acoustic models," in *Proc. Int. Conf. Mach. Learn.*, 2013, vol. 30, no. 1, p. 3.
[46] W. Luo, Y. Li, R. Urtasun, and R. Zemel, "Understanding the effective receptive field in deep convolutional neural networks," 2017, *arXiv:1701.04128*. [Online]. Available: http://arxiv.org/abs/1701.04128
[47] S. Xie, C. Sun, J. Huang, Z. Tu, and K. Murphy, "Rethinking spatiotemporal feature learning: Speed-accuracy trade-offs in video classification," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 305–321.
[48] Z. Qiu, T. Yao, and T. Mei, "Learning spatio-temporal representation with pseudo-3D residual networks," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 5533–5541.
[49] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2818–2826.
[50] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," 2017, *arXiv:1611.01578*. [Online]. Available: https://arxiv.org/abs/1611.01578
[51] S. Chen, Y. Chen, S. Yan, and J. Feng, "Efficient differentiable neural architecture search with meta kernels," 2019, *arXiv:1912.04749*. [Online]. Available: https://arxiv.org/abs/1912.04749
[52] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet V2: Practical guidelines for efficient CNN architecture design," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 116–131.
[53] A. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," *IEEE Trans. Inf. Theory*, vol. IT-13, no. 2, pp. 260–269, Apr. 1967.
[54] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*. [Online]. Available: http://arxiv.org/abs/1412.6980
[55] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. Int. Conf. Learn. Represent.*, 2017, pp. 1–16.
[56] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, "DeepLab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected CRFs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 4, pp. 834–848, Apr. 2018.
[57] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," 2017, *arXiv:1706.05587*. [Online]. Available: http://arxiv.org/abs/1706.05587
[58] A. Paszke *et al.*, "Automatic differentiation in Pytorch," in *Proc. Neural Inf. Process. Syst. (NIPS)*, 2017. [Online]. Available: https://openreview.net/pdf?id=BJJsrmfCZ
[59] W. Hu, Y. Huang, L. Wei, F. Zhang, and H. Li, "Deep convolutional neural networks for hyperspectral image classification," *J. Sensors*, vol. 2015, Jan. 2015, Art. no. 258619.



Haokui Zhang received the M.S. and Ph.D. degrees in computer application technology from the Shannxi Provincial Key Laboratory of Speech and Image Information Processing, Xi'an, China, in 2016 and 2021, respectively.

He is working as an Algorithm Engineer with AI Lab of Intellifusion, Shenzhen, China. His research interests cover approximate nearest neighbor search, semantic segmentation, depth estimation, neural architecture search (NAS), and hyperspectral image classification.



Chengrong Gong is pursuing the Ph.D. degree with the School of Computer Science, Northwestern Polytechnical University, Xi'an, China.

His research interests include hyperspectral image super resolution, image fusion, and deep learning (DL).



Yunpeng Bai received the B.Sc. degree from the Harbin Institute of Technology, Harbin, China, in 2018, and the M.Eng. degree in information systems from the Melbourne School of Engineering, The University of Melbourne, Parkville, VIC, Australia, in 2020.

He is working as a Research Assistant Position with Northwestern Polytechnic University, Xi'an, China, during which he has been involved in several research projects on remote sensing satellite image analysis and object detection. Since 2019, his research interests are in computer vision, with a focus on applied artificial intelligence and deep learning.



Zongwen Bai received the M.S. degree from Yan'an University, Yan'an, China, in 2008. He is pursuing the Ph.D. degree with the School of Computer Science, Northwestern Polytechnical University, Xi'an, China.

He is an Associate Professor with the School of Physics and Electronic Information, Yan'an University. His research interests include hyperspectral image super resolution, image fusion, and deep learning.



Ying Li received the Ph.D. degree in electrical circuit and system from the National Key Laboratory of Radar Signal Processing, Xidian University, Xi'an, China, in 2002.

She is a Professor with the School of Computer Science, Northwestern Polytechnical University, Xi'an. Her interests cover image processing, computation intelligence, and signal processing.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 10:42:04 UTC from IEEE Xplore. Restrictions apply.